

Self-Attention

Rasha Obeidat, PhD

NLP

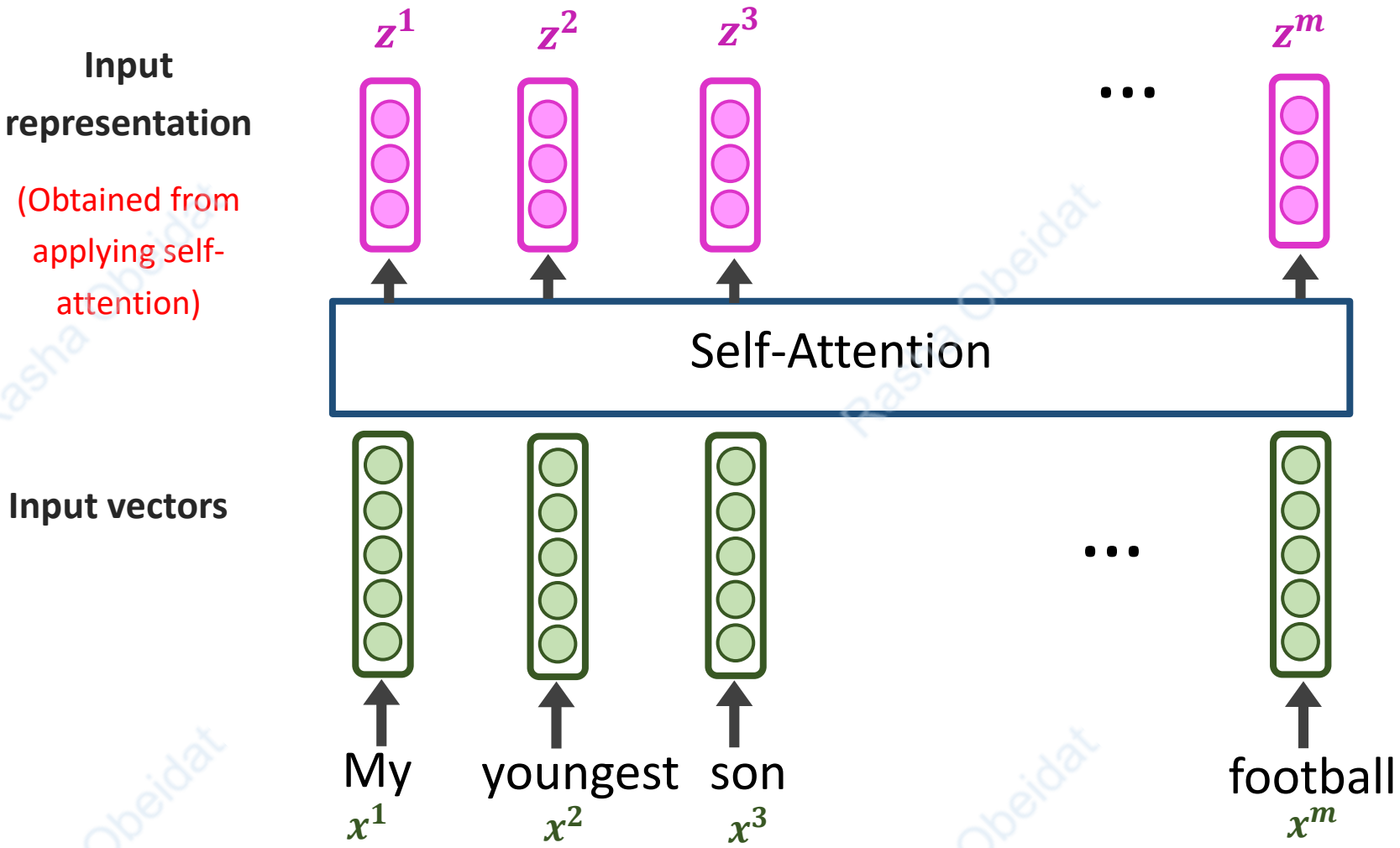
Rasha Obeidat

Self-Attention: Definition

- **What is Self-Attention?**
  - A mechanism to determine how each word (or token) in a sequence relates to other words in *the same sequence*.
  - Enables models to weigh the importance of different surrounding words when generating context-aware representation of each word.
  - Also Known as ***intra-attention***
- **Key Idea**:
  - Instead of processing words independently, self-attention considers the entire sequence to create richer embeddings for each word.

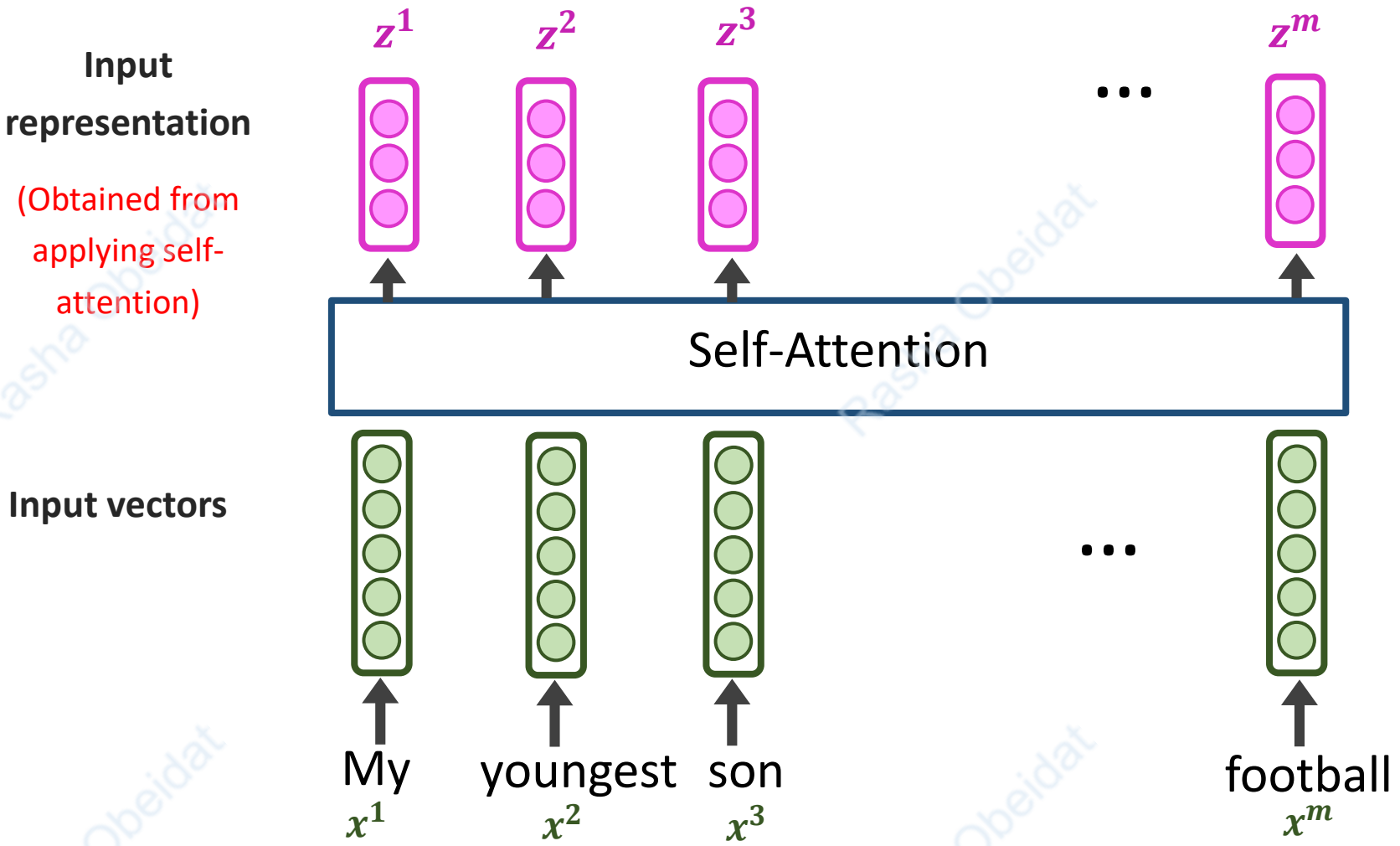
Dr. Rasha Obeidat®

Self-Attention



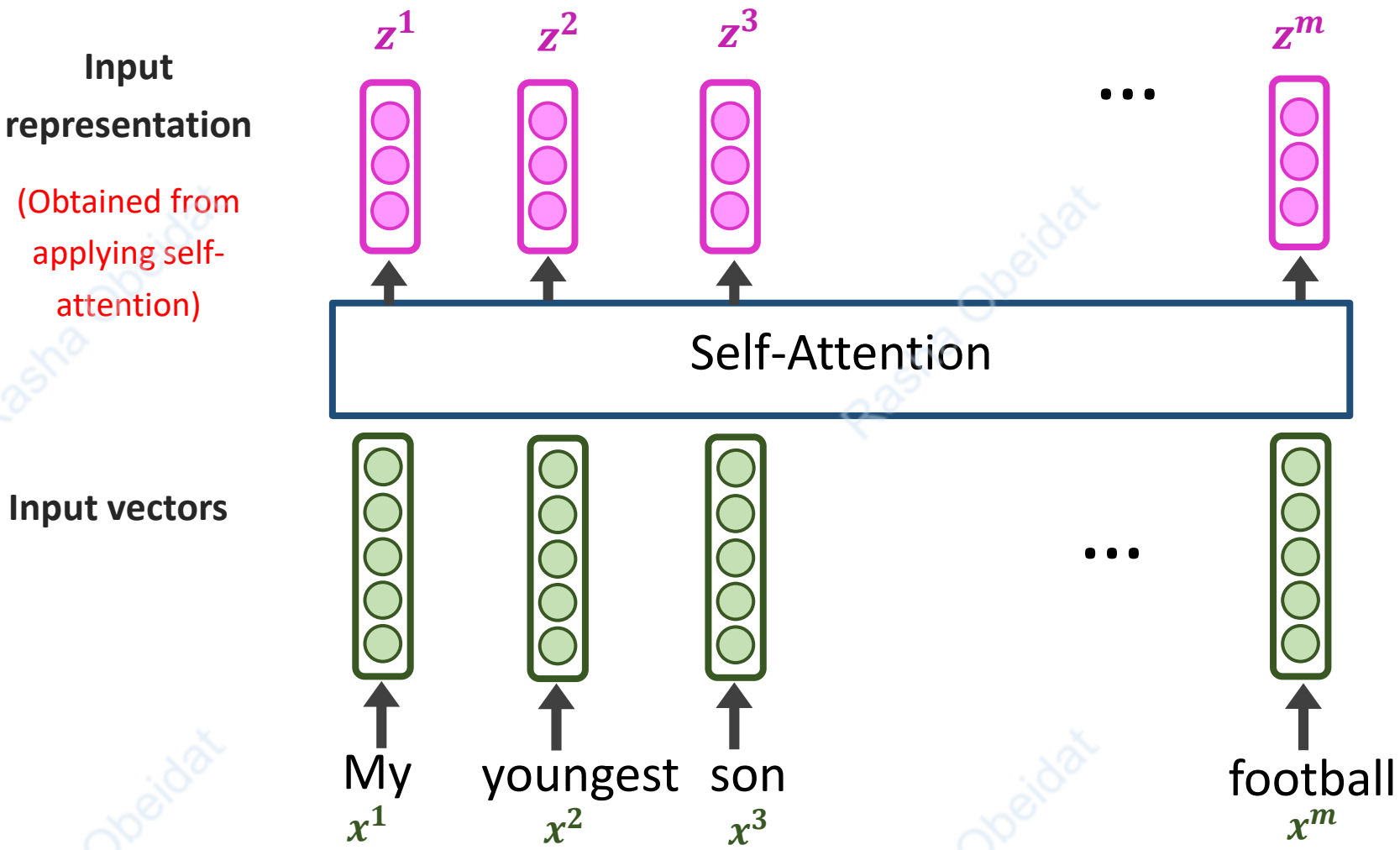
Self-Attention's goal is to create context-aware representations, z^i, of the input

Self-Attention



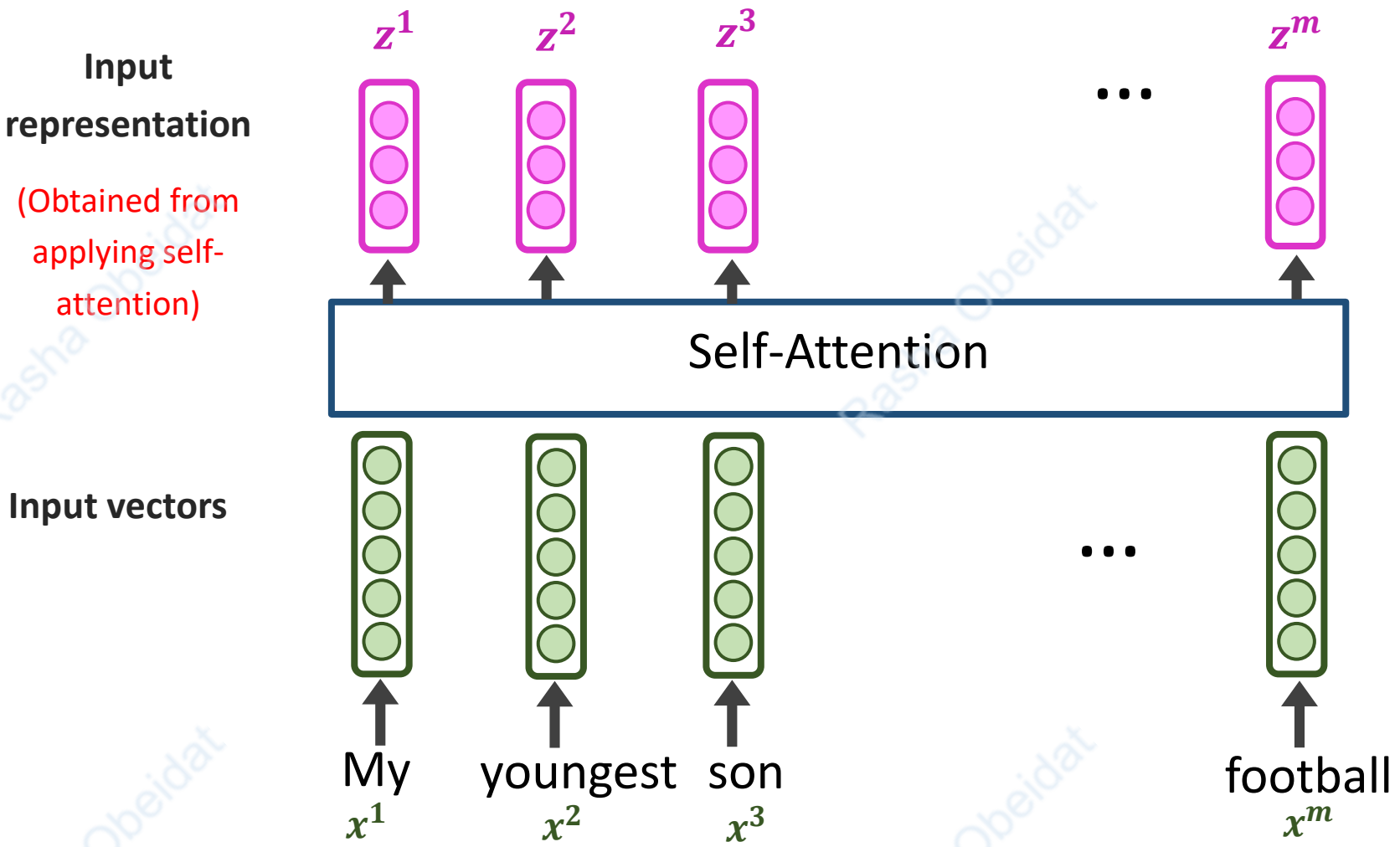
- **Self-Attention**'s goal is to create high-quality representations, z^i, of the input
- Each z^i will be based on a weighted contribution of x^1, $x^2, x^3 \ldots x^m$

Self-Attention



- **Self-Attention**'s goal is to create high-quality representations, z^i, of the input
- Each z^i will be based on a weighted contribution of x^1, x^2,x^3 ...x^m
- How?!

Self-Attention

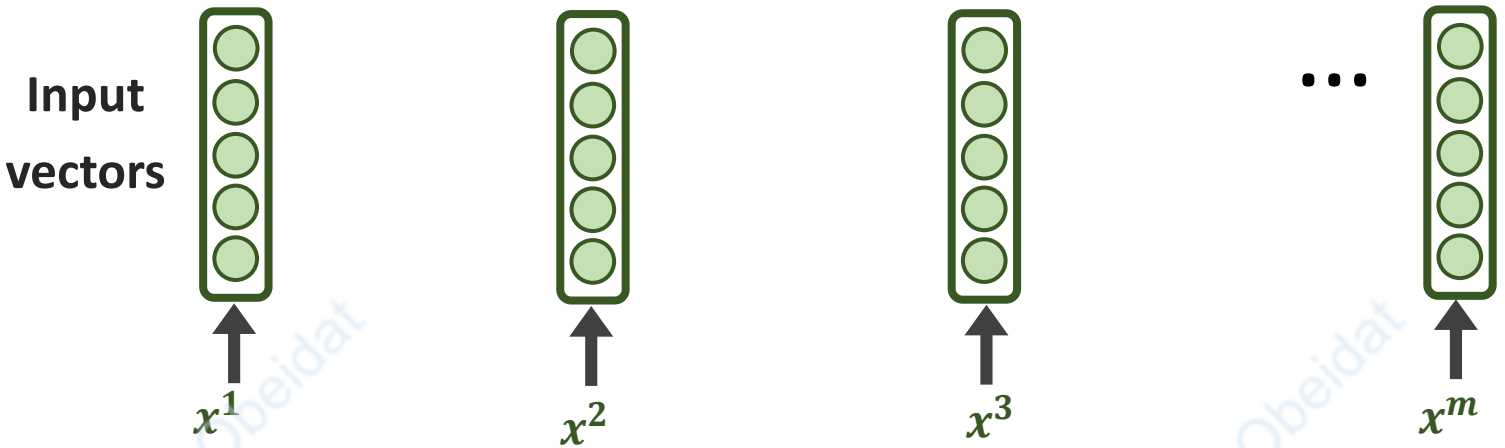


- **Self-Attention**'s goal is to create high-quality representations, z^i, of the input
- Each z^i will be based on a weighted contribution of x^1, x^2,x^3 ...x^m
- How?!

Self-Attention

Step1: Input Representation:

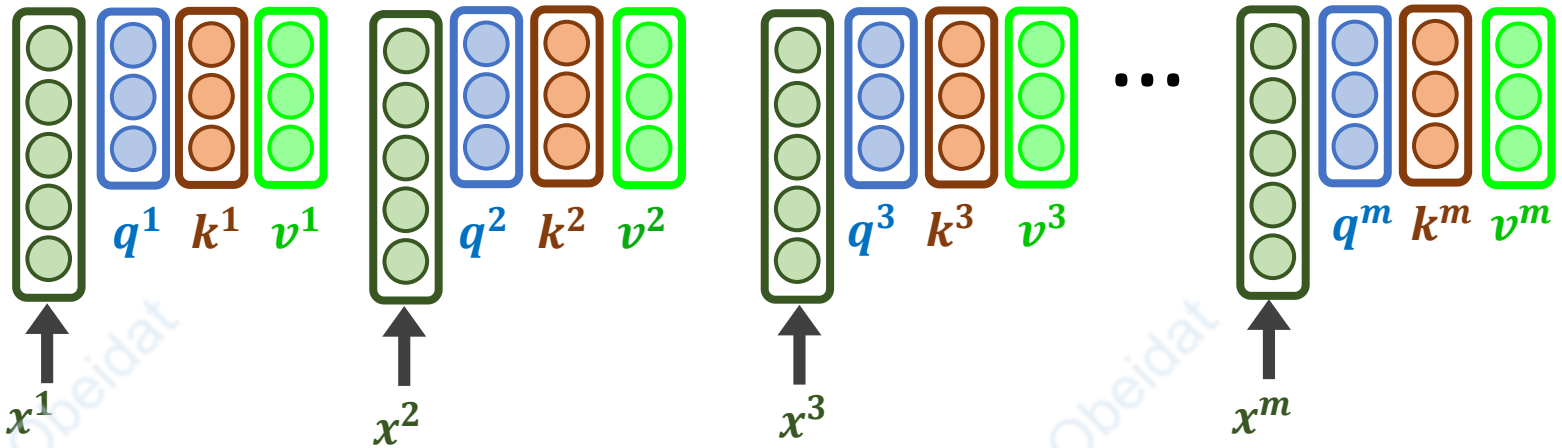
- The input is typically a sequence of tokens (e.g., words in a sentence).
- Each token is represented as a vector, usually from an embedding layer.



Self-Attention

Step2: Linear Transformations

- Each x^i has 3 small, associated vectors.
  - Query q^i
  - Key k^i
  - Value v^i

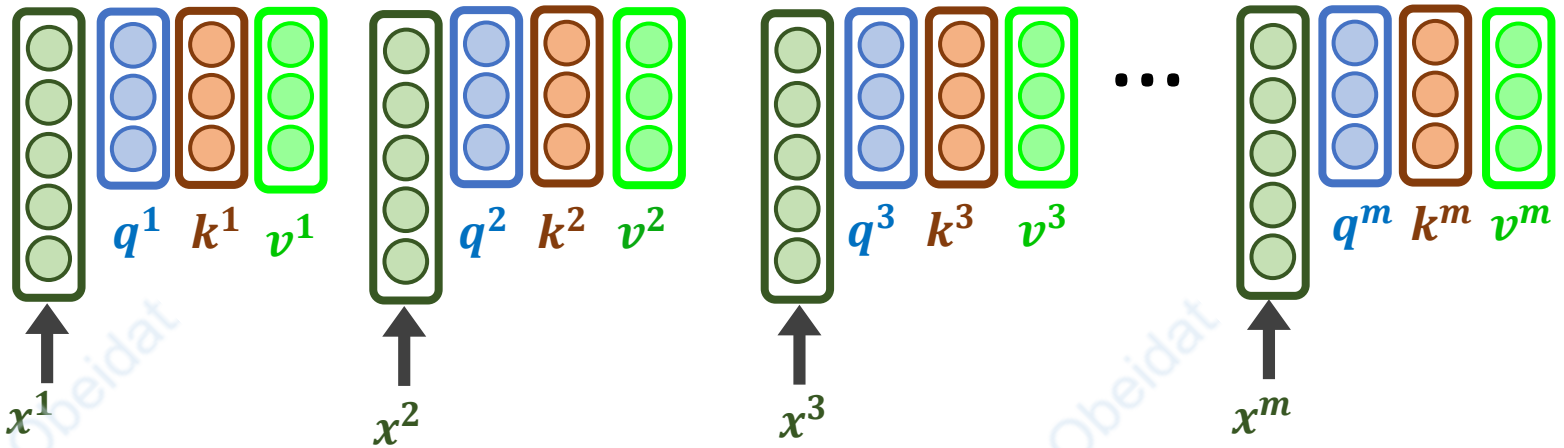


Self-Attention

- The **Query** and the **Key** vectors must all have equal dimensions (d^k)
- Value vectors can have a different dimensions than the query and the key (d^v)
- The presentation z (that results from applying self attention to the input x) will have the same dimensions as the value vector (which is d^v)

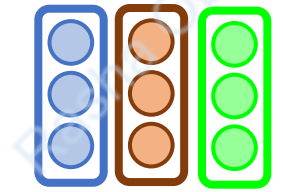
Step2: Linear Transformations

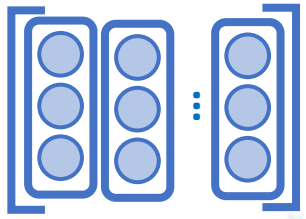
- Each x^i has 3 small, associated vectors.
  - Query $q^i \in R^{d^k}$
  - Key $k^i \in R^{d^k}$
  - Value $v^i \in R^{d^v}$
- A basic Self-Attention Head has just 3 weight matrices W_q, W_k, W_v. These ***same*** 3 weight matrices are multiplied by each x^i to create q^i, k^i and v^i vectors for each time step i.
- Computed as follows:
  - $q^i = W_q x^i$
  - $k^i = W_k x^i$
  - $v^i = W_v x^i$

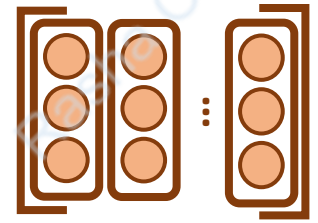


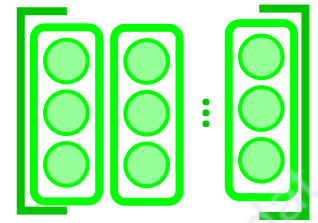
Why Use Keys, Queries, and Values?

- **What They Represent**:
  - These are **stand-ins** and simplified versions (or "proxies“) for the actual data.
    - **Query (Q):** The token being focused on; the probe.
    - **Key (K):** The tokens being compared to; determines relevance.
    - **Value (V):** The information associated with each token; what gets weighted and passed forward.



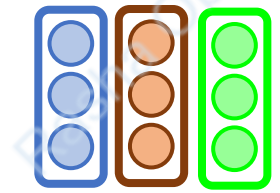


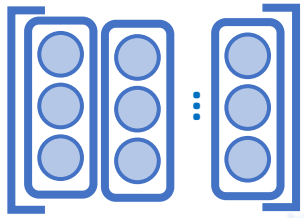


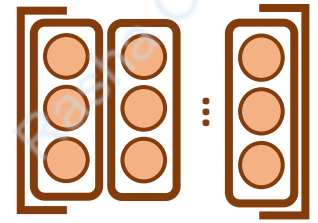


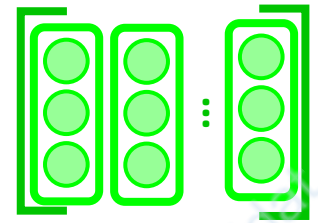
Why Use Keys, Queries, and Values?

- **Importance Over Raw Input**:
  - **Efficient Focus on relationships:** Q, K, and V allow the model to dynamically compute relationships between tokens without relying on raw input directly.
  - **Enable rich abstractions:** These representations capture *essential features* of the data while ignoring unnecessary details.
  - **Dimensional flexibility and fewer constraints:** Transformations into Q, K, and V adapt input into suitable dimensions for computational efficiency and help capture complex token interactions without being tied to the raw sequence structure.
  - **Rich contextual representations:** Enable the model to weigh the importance of each token dynamically, resulting in better embeddings for downstream tasks.







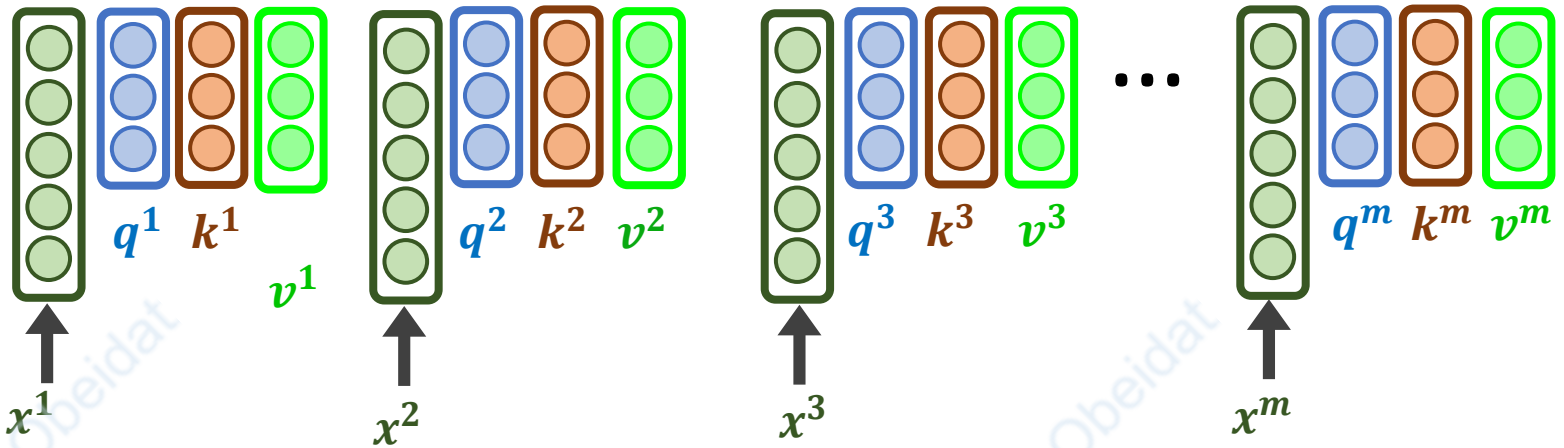


Self-Attention

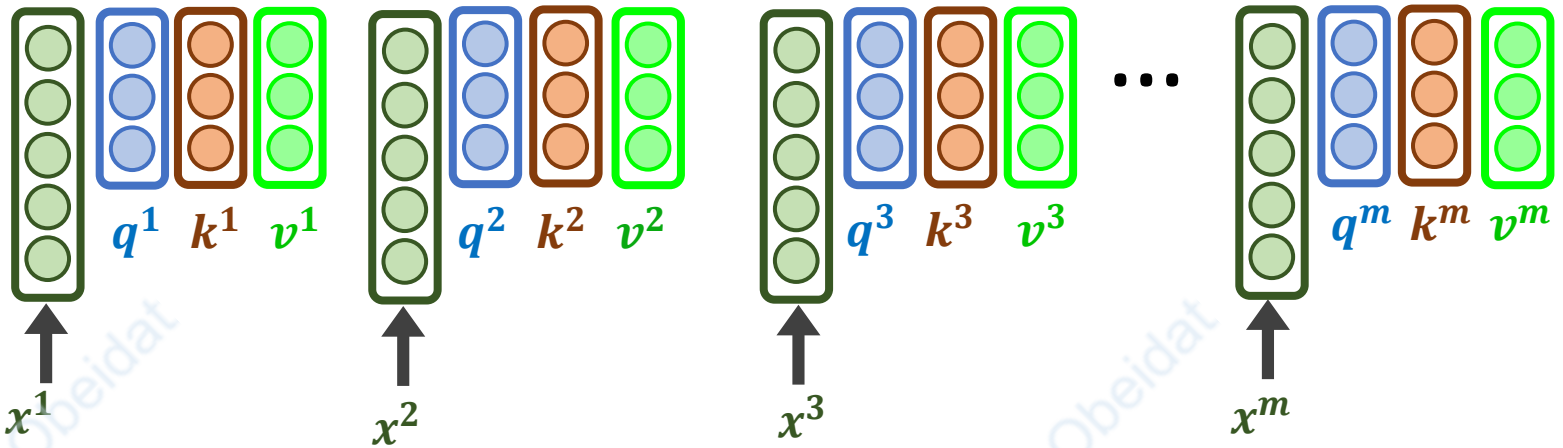
- The **Query** and the **Key** vectors must all have equal dimensions (d^k)
- Value vectors can have a different dimensions than the query and the key (d^v)
- The presentation Z (that results from applying self attention to the input x) will have the same dimensions as the value vector (which is d^v)

Step2: Linear Transformations

- Each x^i has 3 small, associated vectors.
  - Query $q^i \in R^{d^k}$
  - Key $k^i \in R^{d^k}$
  - Value $v^i \in R^{d^v}$
- A basic Self-Attention Head has just 3 weight matrices W_q, W_k, W_v. These ***same*** 3 weight matrices are multiplied by each x^i to create q^i, k^i and v^i vectors for each time step i.
- Computed as follows:
  - $q^i = W_q x^i$
  - $k^i = W_k x^i$
  - $v^i = W_v x^i$



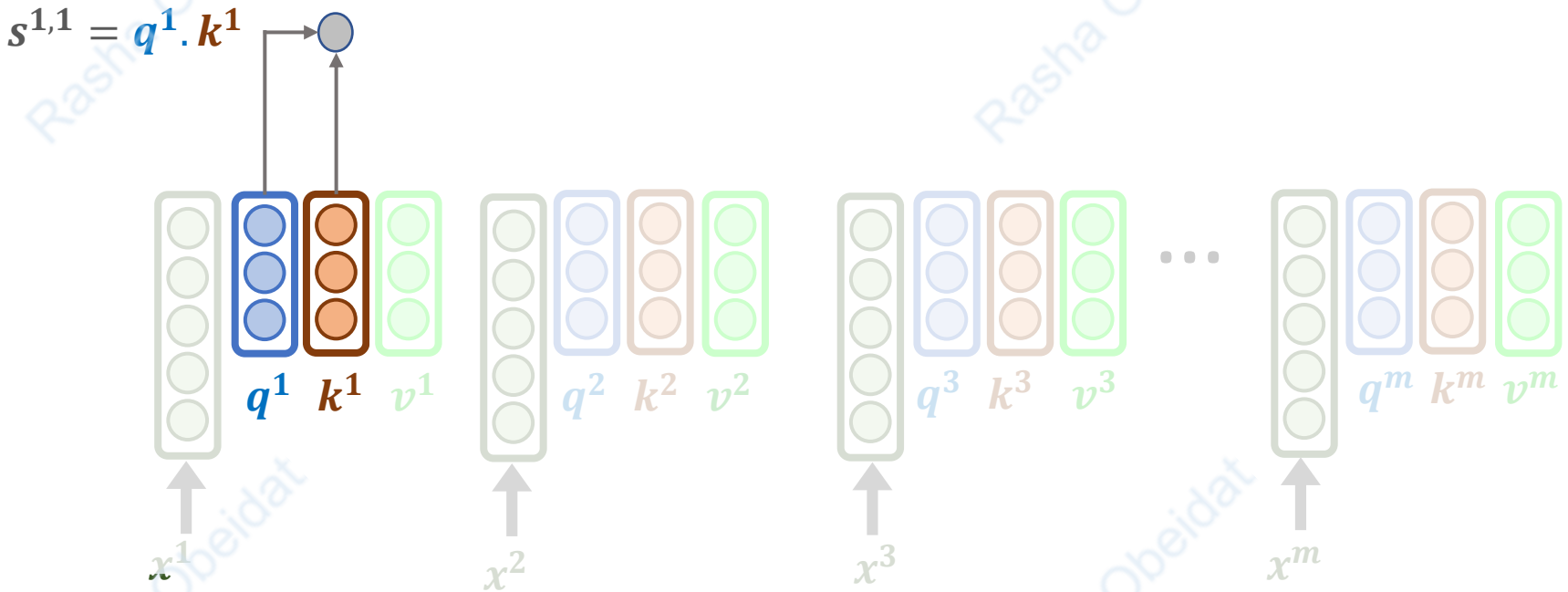
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

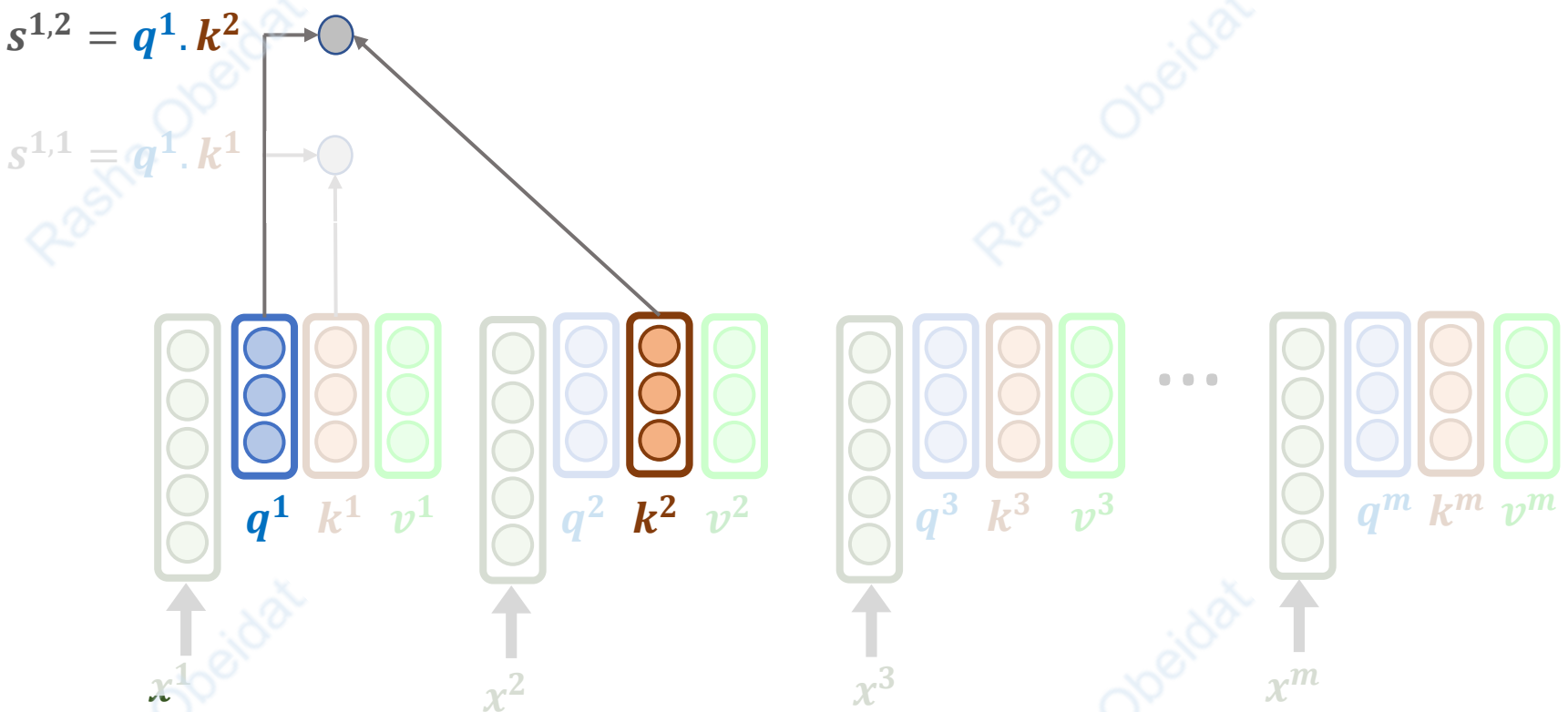
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \ldots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \ldots, s^{i,m}$ for each $i = 1, 2, \ldots, m$.
- This means that a similarity score is computed for each pair of tokens

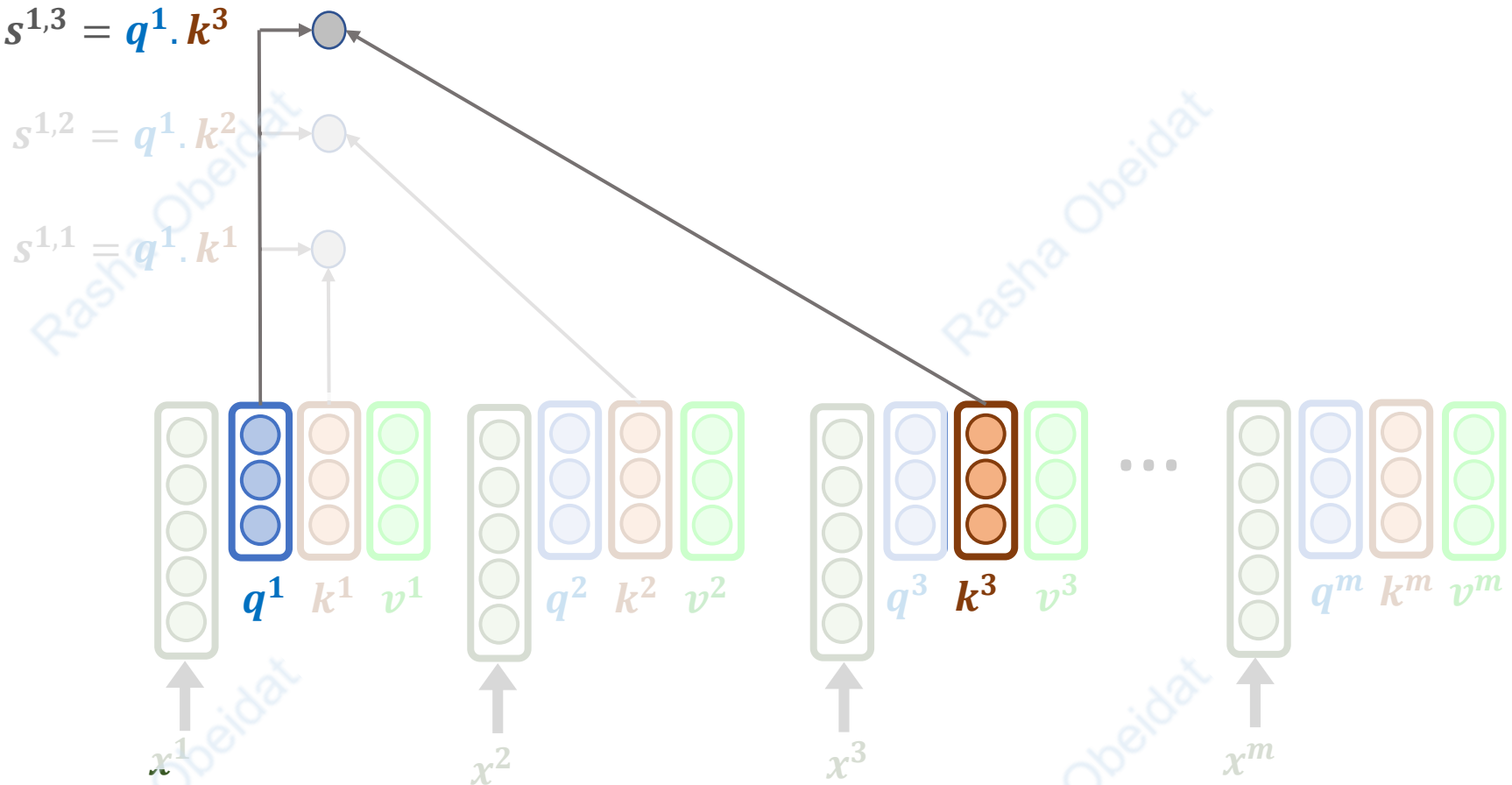
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

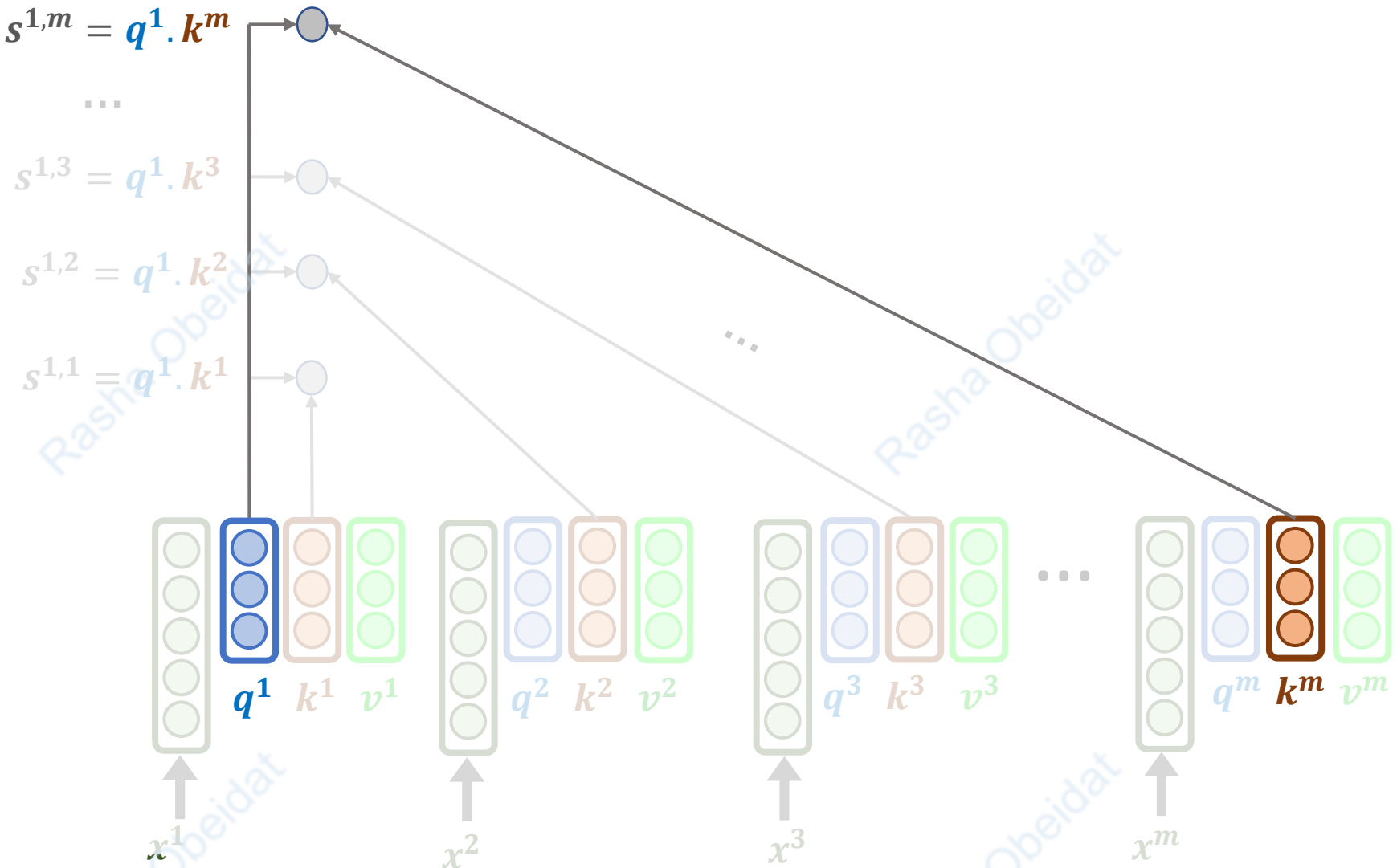
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

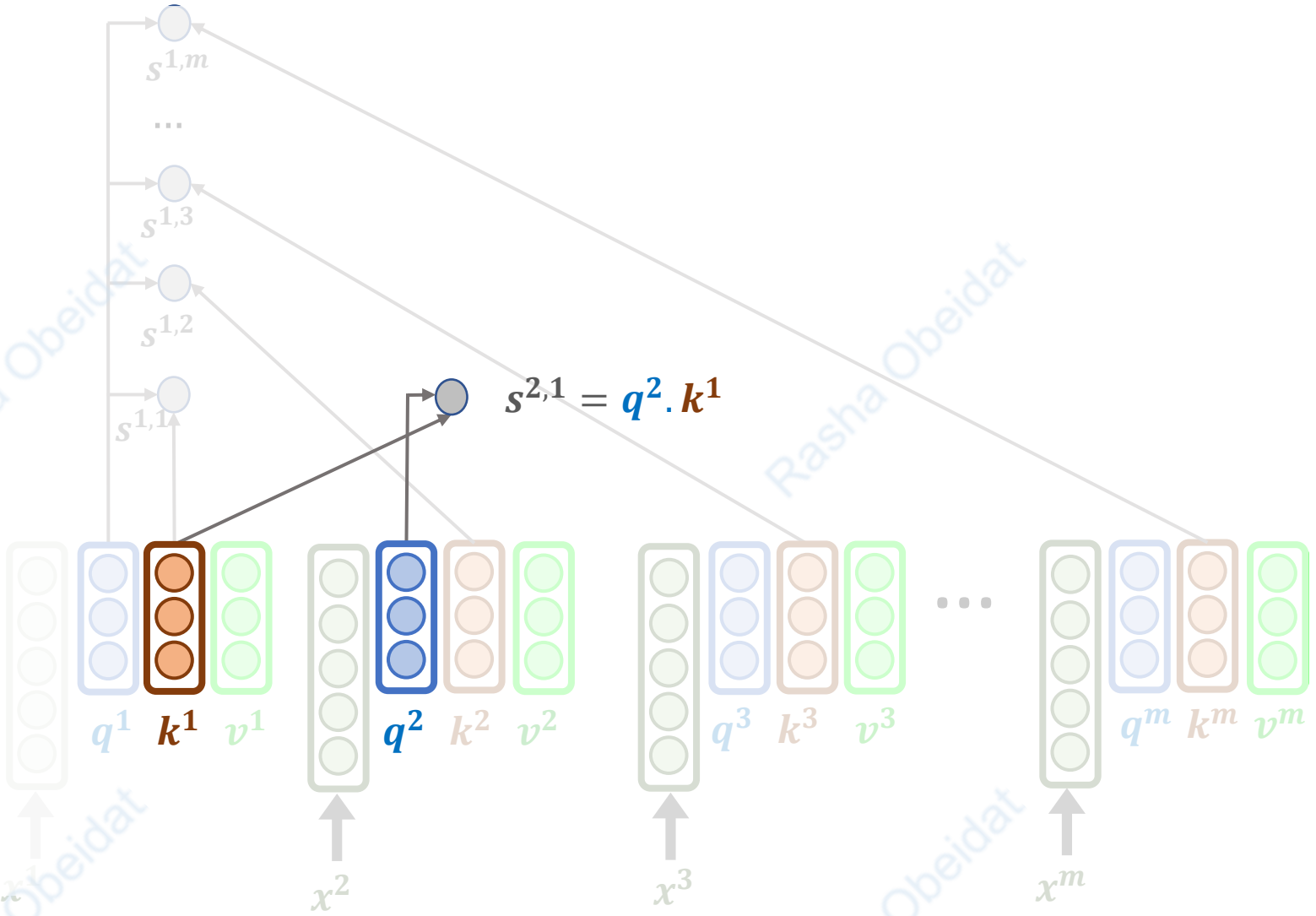
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \ldots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \ldots, s^{i,m}$ for each $i = 1, 2, \ldots, m$.
- This means that a similarity score is computed for each pair of tokens

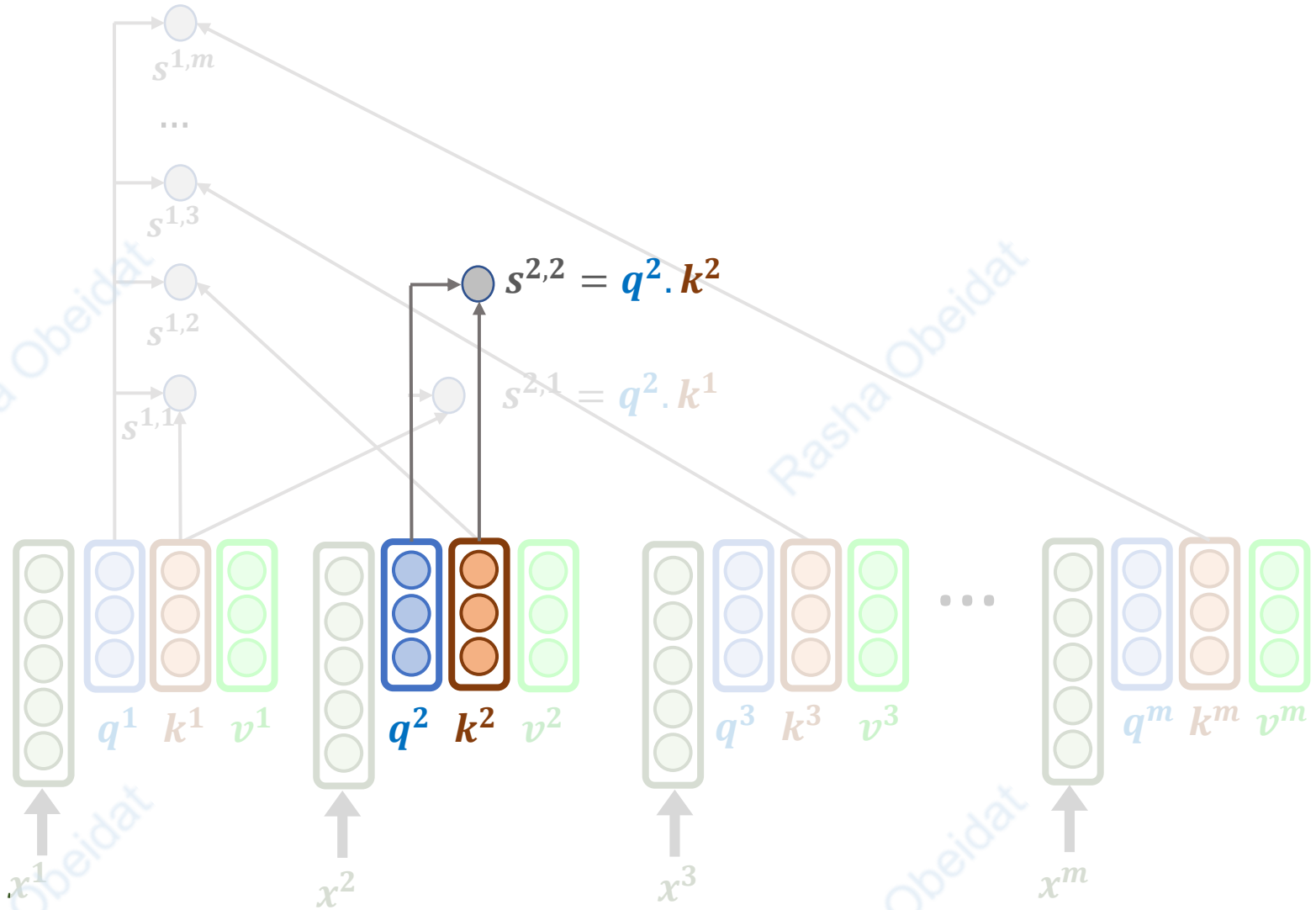
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

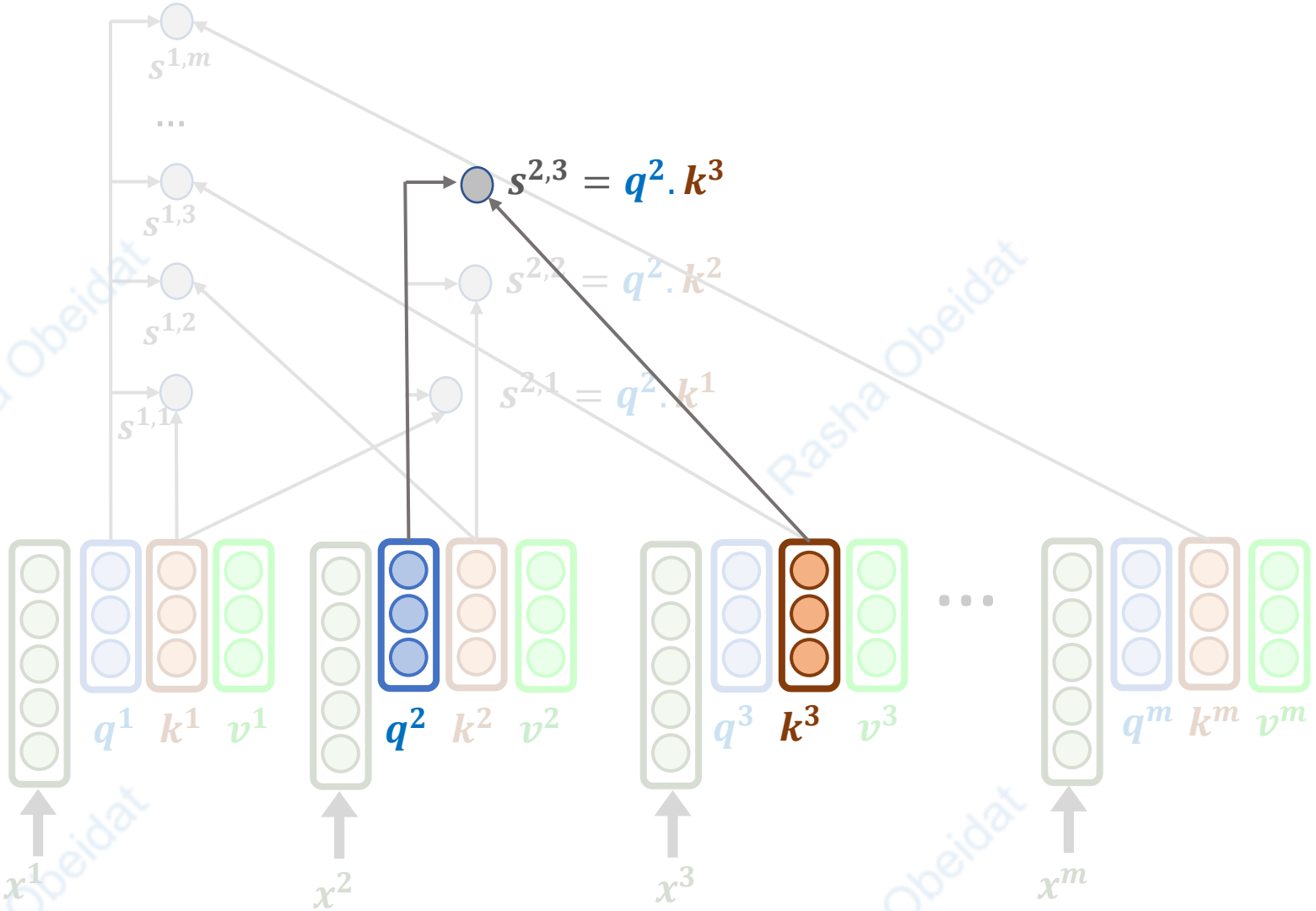
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

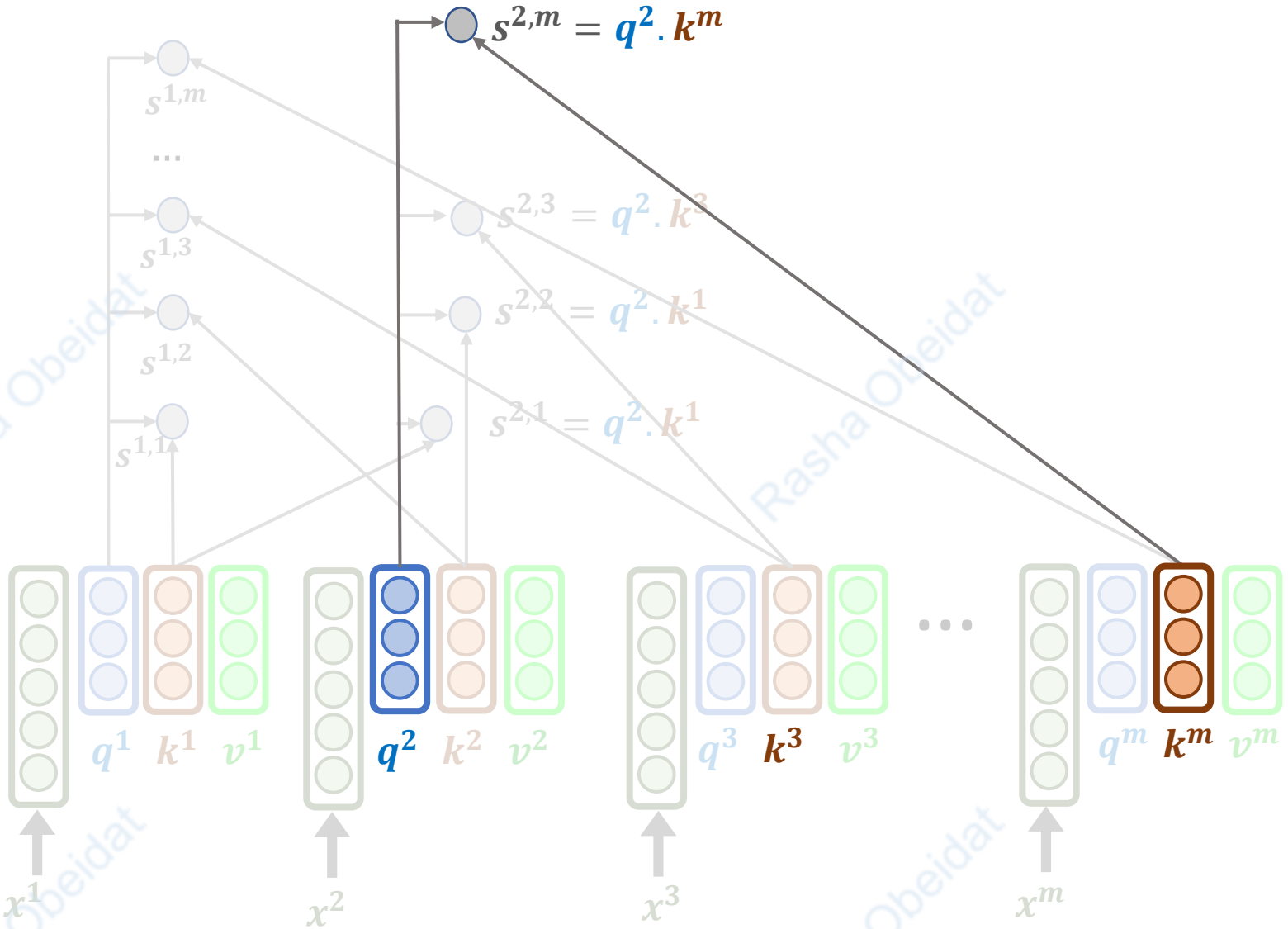
Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

Self-Attention



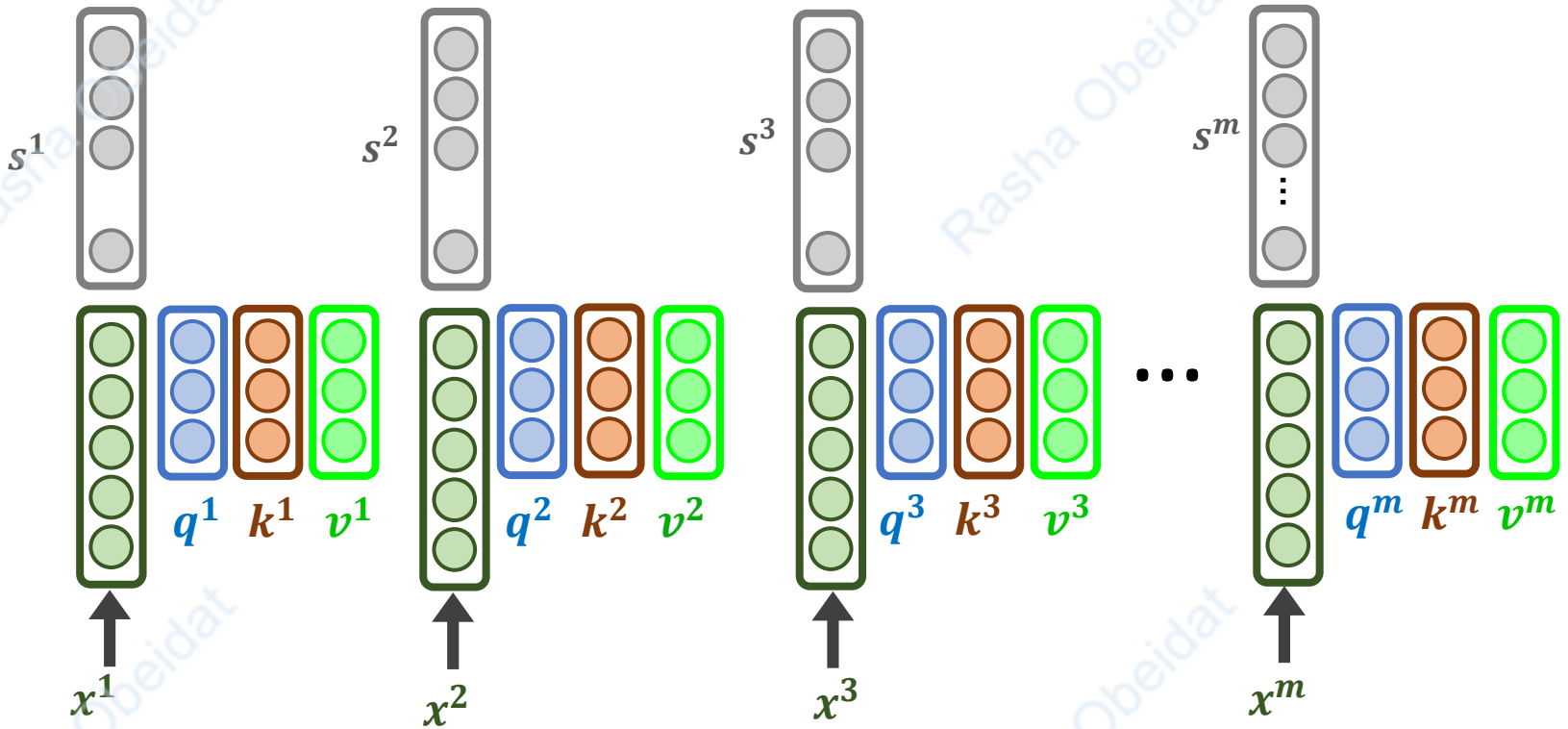
Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

and so on….

Self-Attention

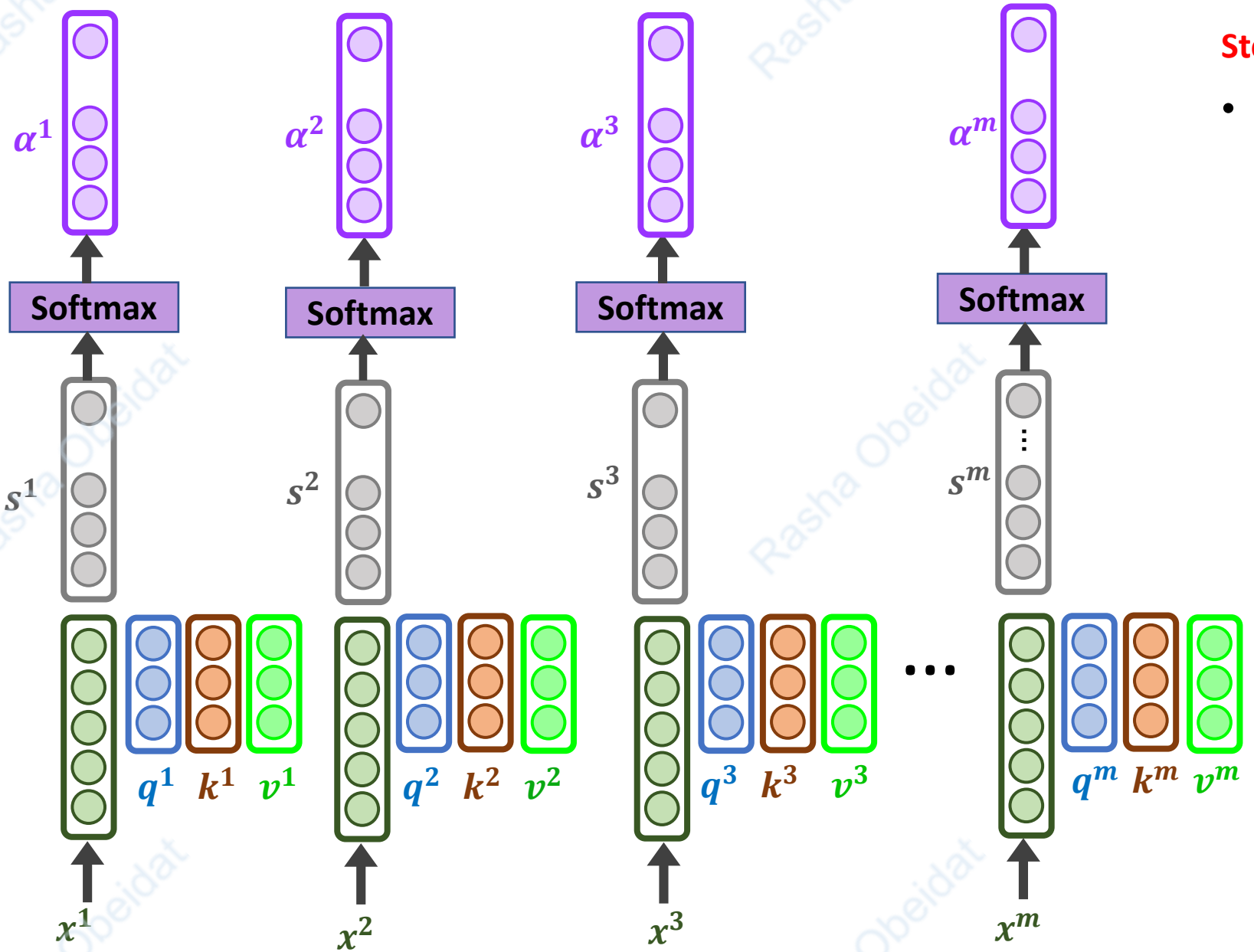
$$s^1 = [s^{1,1}, s^{1,2}, s^{1,3}, \ldots, s^{1,m}]^T$$
$$s^2 = [s^{2,1}, s^{2,2}, s^{2,3}, \ldots, s^{2,m}]^T$$
$$s^3 = [s^{3,1}, s^{3,2}, s^{3,3}, \ldots, s^{3,m}]^T$$
$$\ldots$$
$$s^m = [s^{m,1}, s^{m,2}, s^{m,3}, \ldots, s^{m,m}]^T$$



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \ldots, m$ in the sequence using a dot product.
- So, for each token x^1, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,1}, \ldots, s^{i,m}$ for each $i = 1, 2, \ldots, m$.
- This means that a similarity score is computed for each pair of tokens

Self-Attention



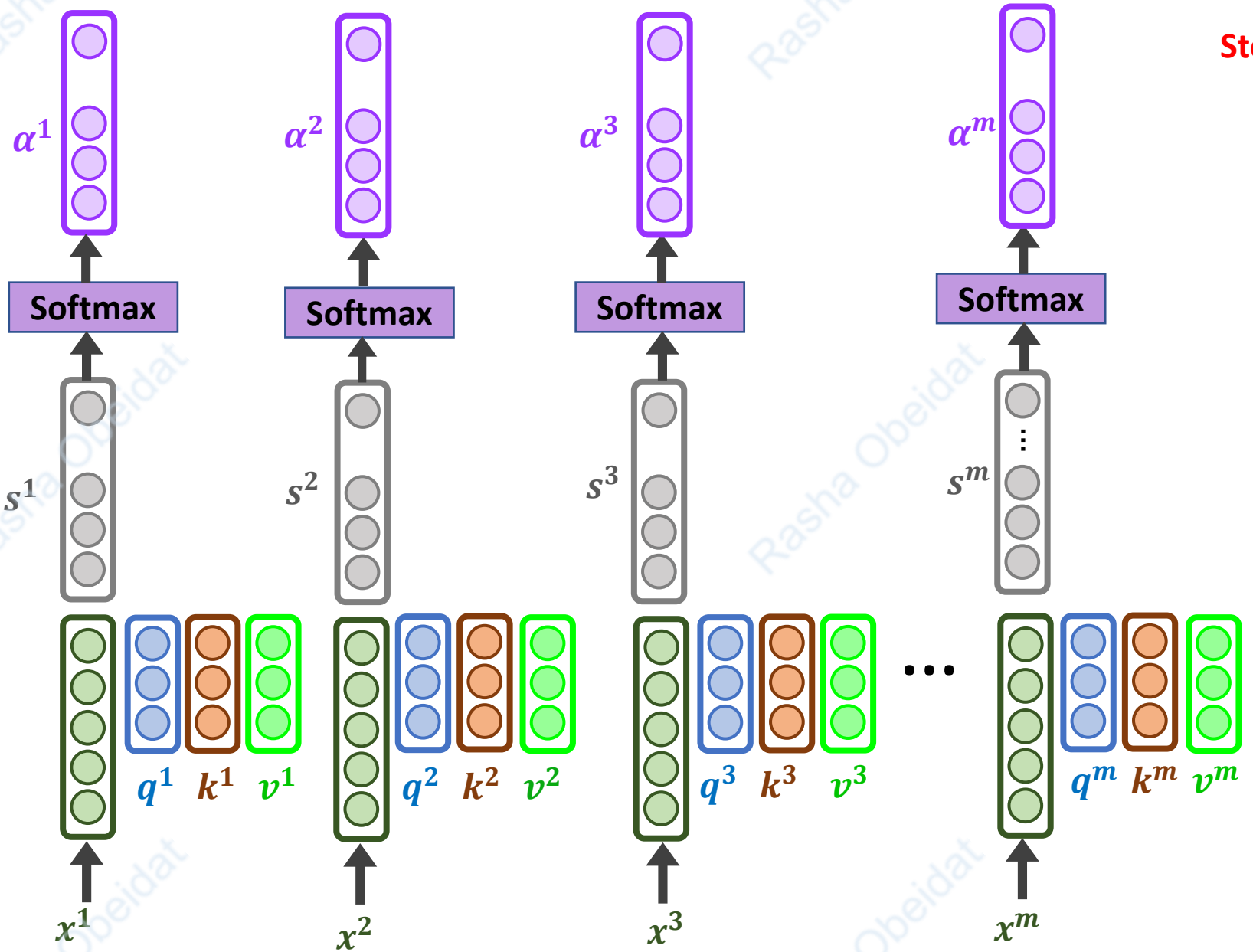
Step4: Compute Softmax

- The similarity scores are passed through a **softmax function** to normalize them into probabilities (attention weights), ensuring they sum to 1.

$$\alpha^1 = [\alpha^{1,1}, \alpha^{1,2}, \alpha^{1,3}, \ldots, \alpha^{1,m}]^T$$
$$\alpha^2 = [\alpha^{2,1}, \alpha^{2,2}, \alpha^{2,3}, \ldots, \alpha^{2,m}]^T$$
$$\alpha^3 = [\alpha^{3,1}, \alpha^{3,2}, \alpha^{3,3}, \ldots, \alpha^{3,m}]^T$$
$$\alpha^m = [\alpha^{m,1}, \alpha^{m,2}, \alpha^{m,3}, \ldots, \alpha^{m,m}]^T$$

$$\alpha^i = \text{Softmax}\left(\frac{s^i}{\sqrt{d^k}}\right)$$

Self-Attention

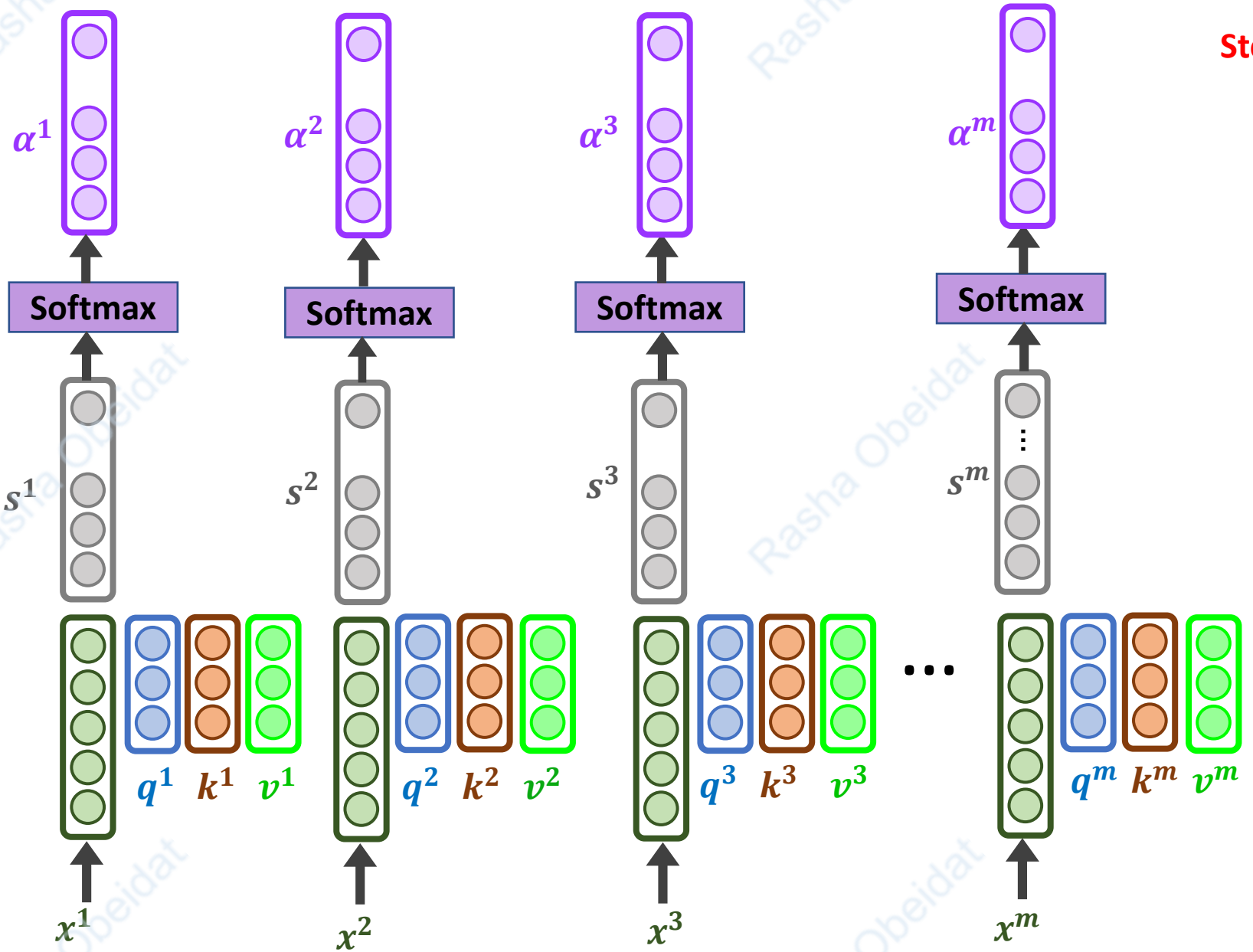


Step4: Compute Softmax

$$\alpha^i = \text{Softmax}\left(\frac{s^i}{\sqrt{d^k}}\right)$$

- large values of d^k, the dot products s^i grow large in magnitude, pushing the Softmax function into regions where it has extremely small gradients .
- To avoid this effect, the dot products are scaled by being multiplied by $\frac{1}{\sqrt{d^k}}$

Self-Attention



Step4: Compute Softmax

$$\alpha^i = \text{Softmax}(\frac{s^i}{\sqrt{d^k}})$$

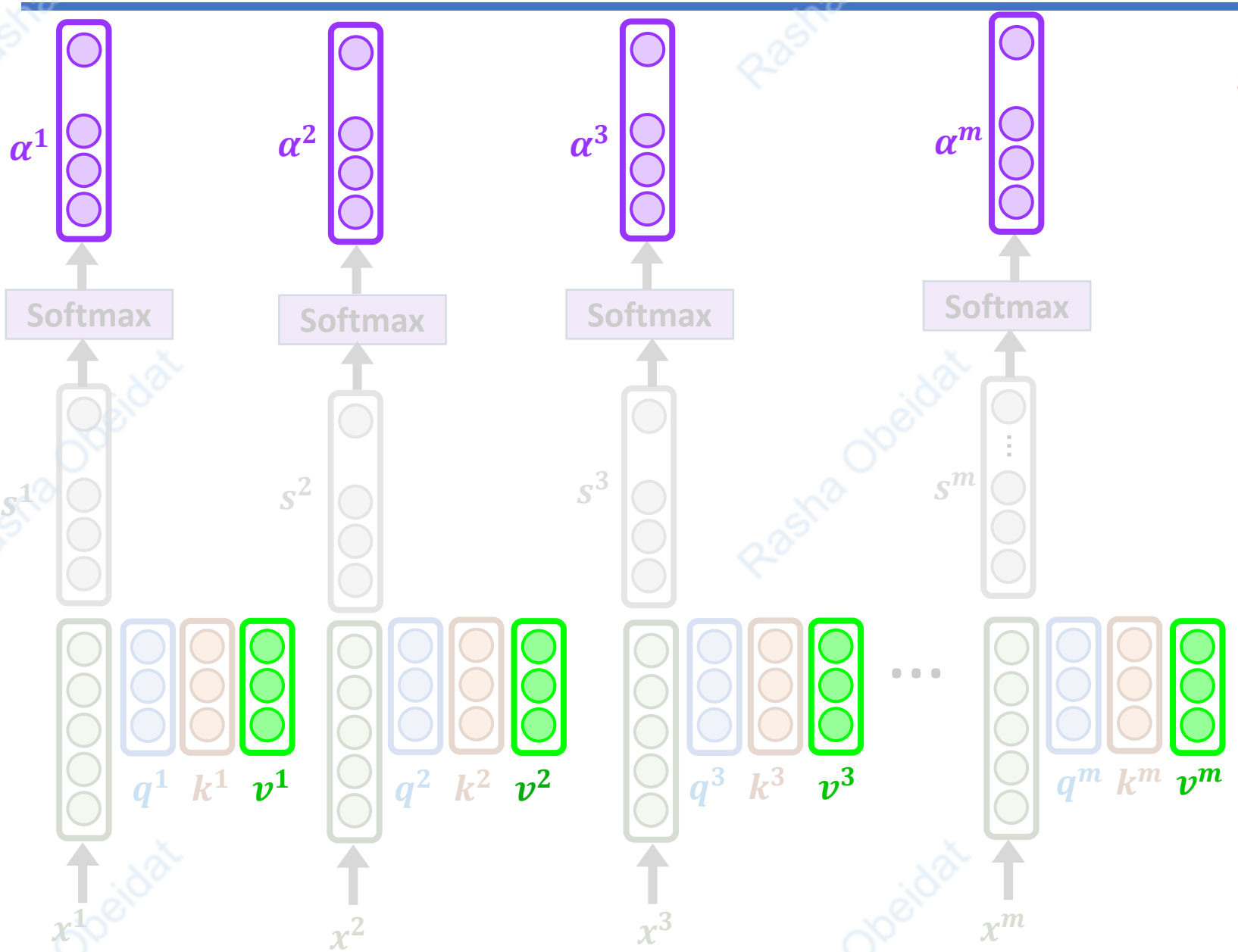
$$s^i = q^i k^i$$

$$\alpha = \text{Softmax}(\frac{S}{\sqrt{d^k}})$$

$$S = QK$$

$$\alpha = \text{Softmax}(\frac{QK}{\sqrt{d^k}})$$

Self-Attention



Step5: Compute Weighted Sum

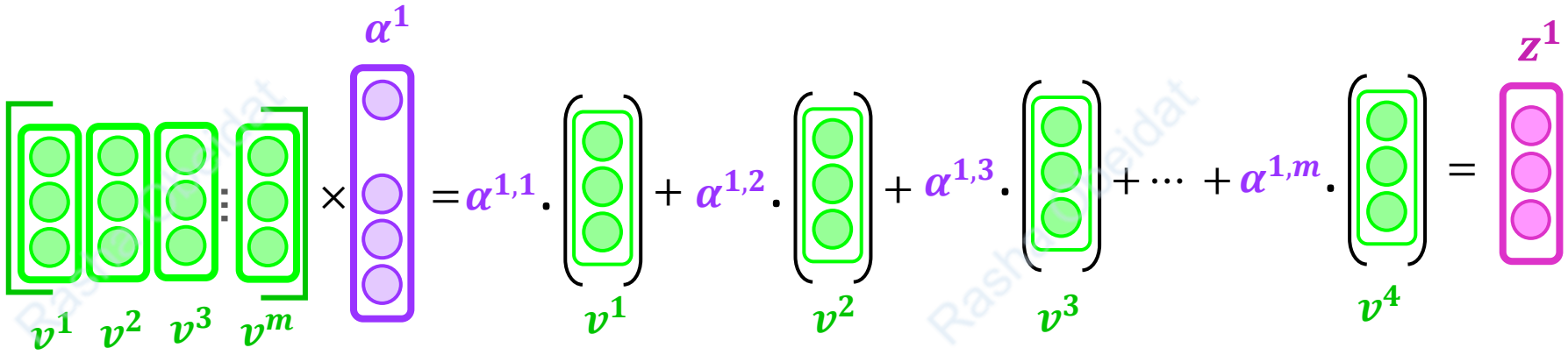
- The attention weights are used to compute a weighted sum of the value vectors vectors ($v^{i,1}$, $v^{i,2}$, ..., $v^{i,m}$) . This sum becomes the output representation z^i for the token x^i.

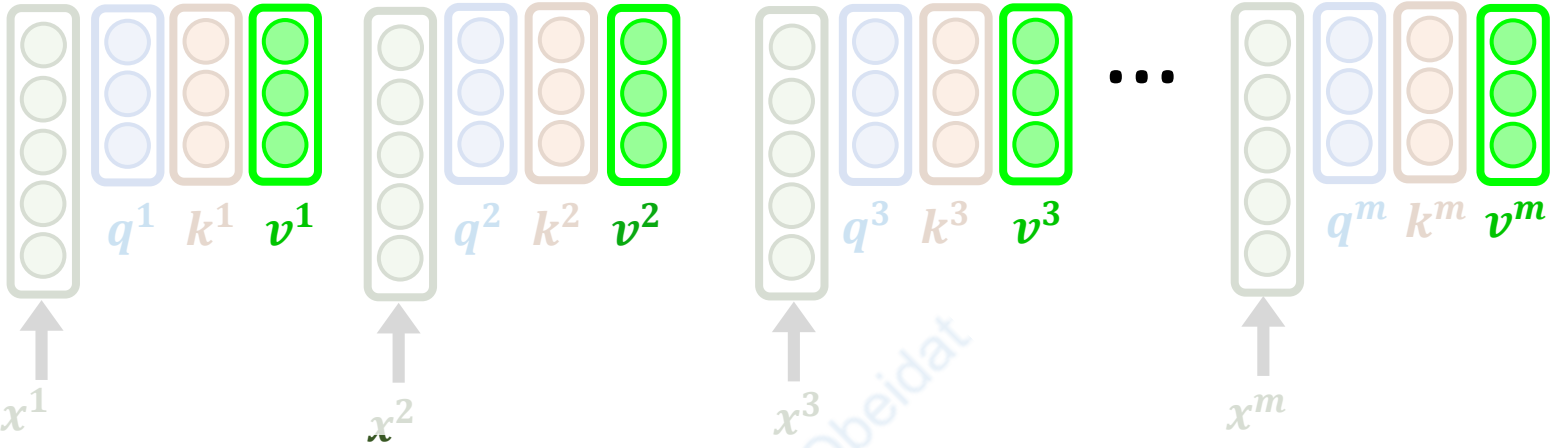
- To get a representation z^i of the token x^i, the attention vector α^i is used to weight the value vectors ($v^{i,1}$, $v^{i,2}$, ..., $v^{i,m}$) vectors Instead directly weighting the original token vectors ($x^{i,1}$, $x^{i,2}$, ..., $x^{i,m}$).

Self-Attention

Step5: Compute Weighted Sum

$$z^1 = \alpha^{1,1} . v^1 + \alpha^{1,2} . v^2 + \alpha^{1,3} . v^3 + \cdots + \alpha^{1,m} . v^m$$

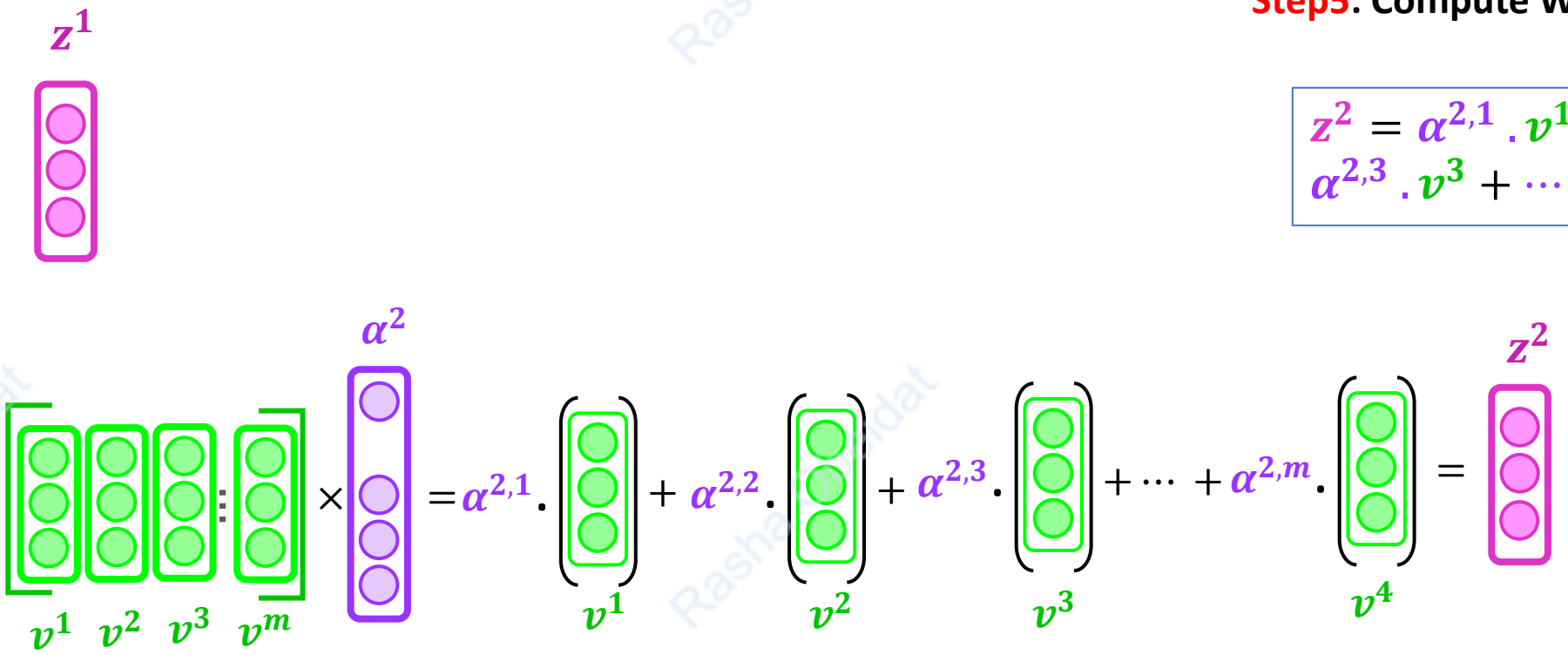




Self-Attention

Step5: Compute Weighted Sum

$$z^2 = \alpha^{2,1} . v^1 + \alpha^{2,2} . v^2 + \alpha^{2,3} . v^3 + \cdots + \alpha^{2,m} . v^m$$

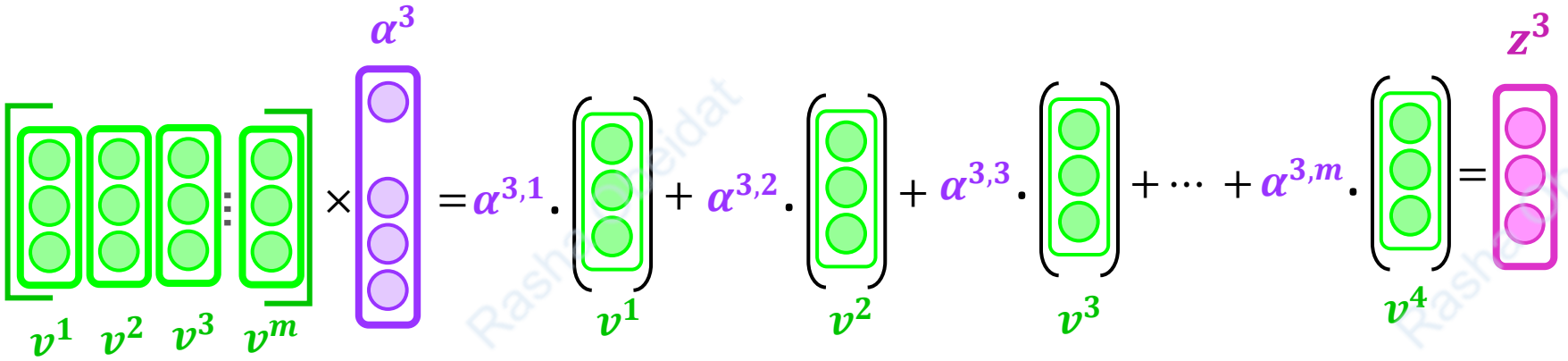


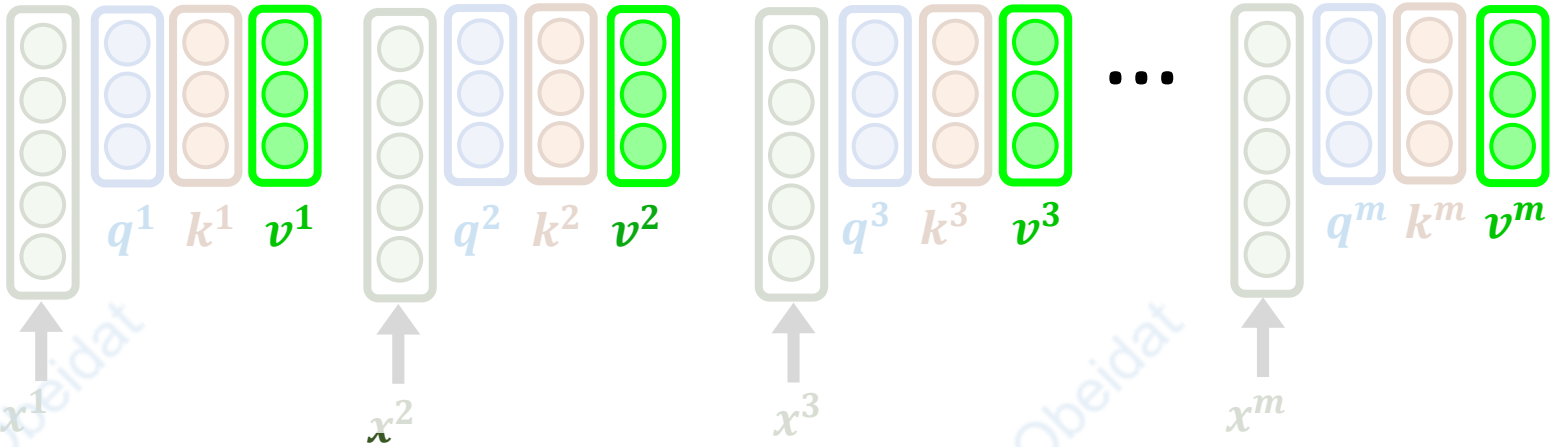
Self-Attention

Step5: Compute Weighted Sum

z^1 z^2

$$z^3 = \alpha^{3,1} . v^1 + \alpha^{3,2} . v^2 + \alpha^{3,3} . v^3 + \cdots + \alpha^{3,m} . v^m$$



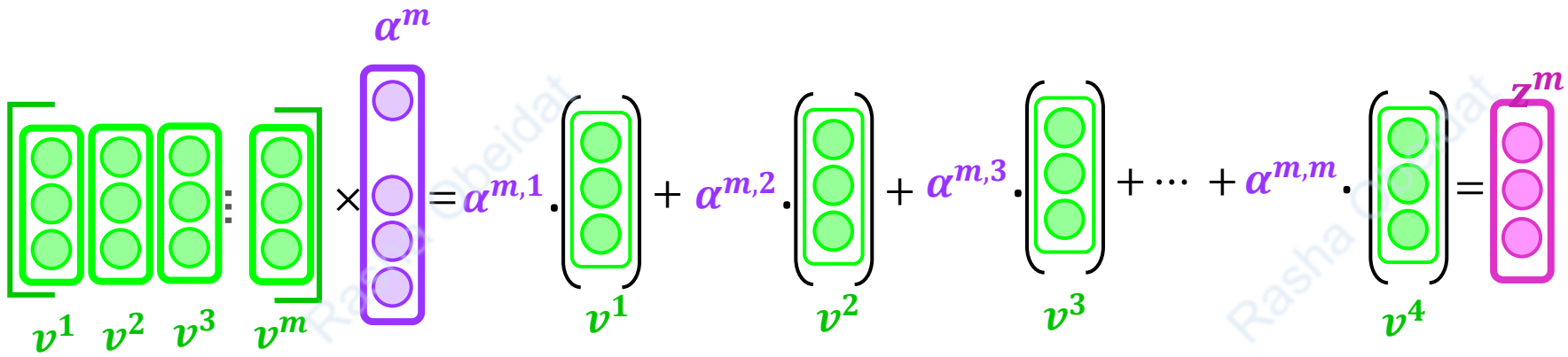


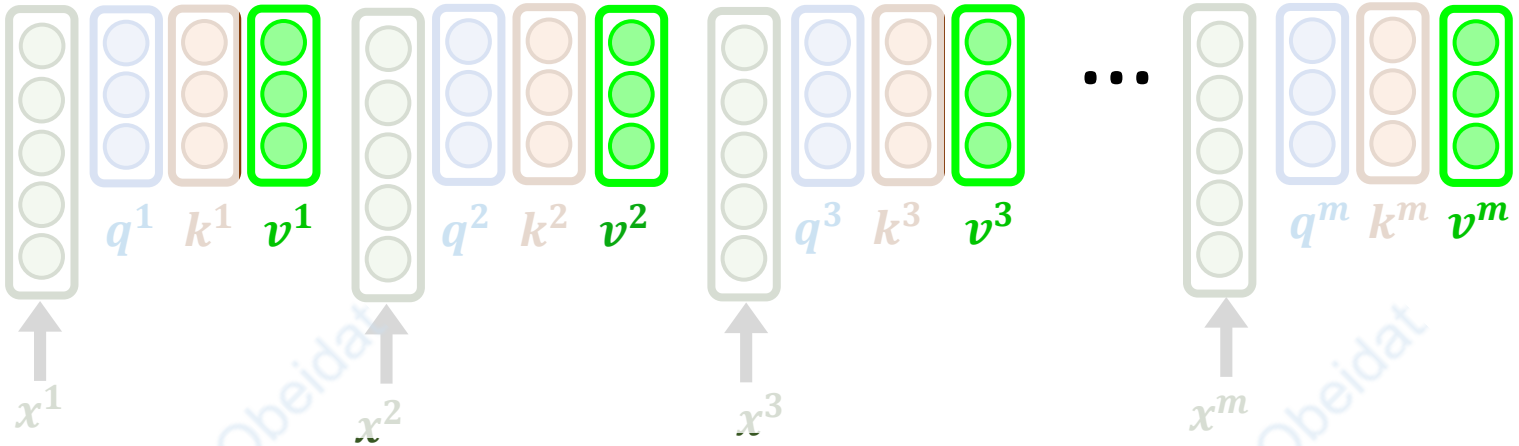
Self-Attention

Step5: Compute Weighted Sum

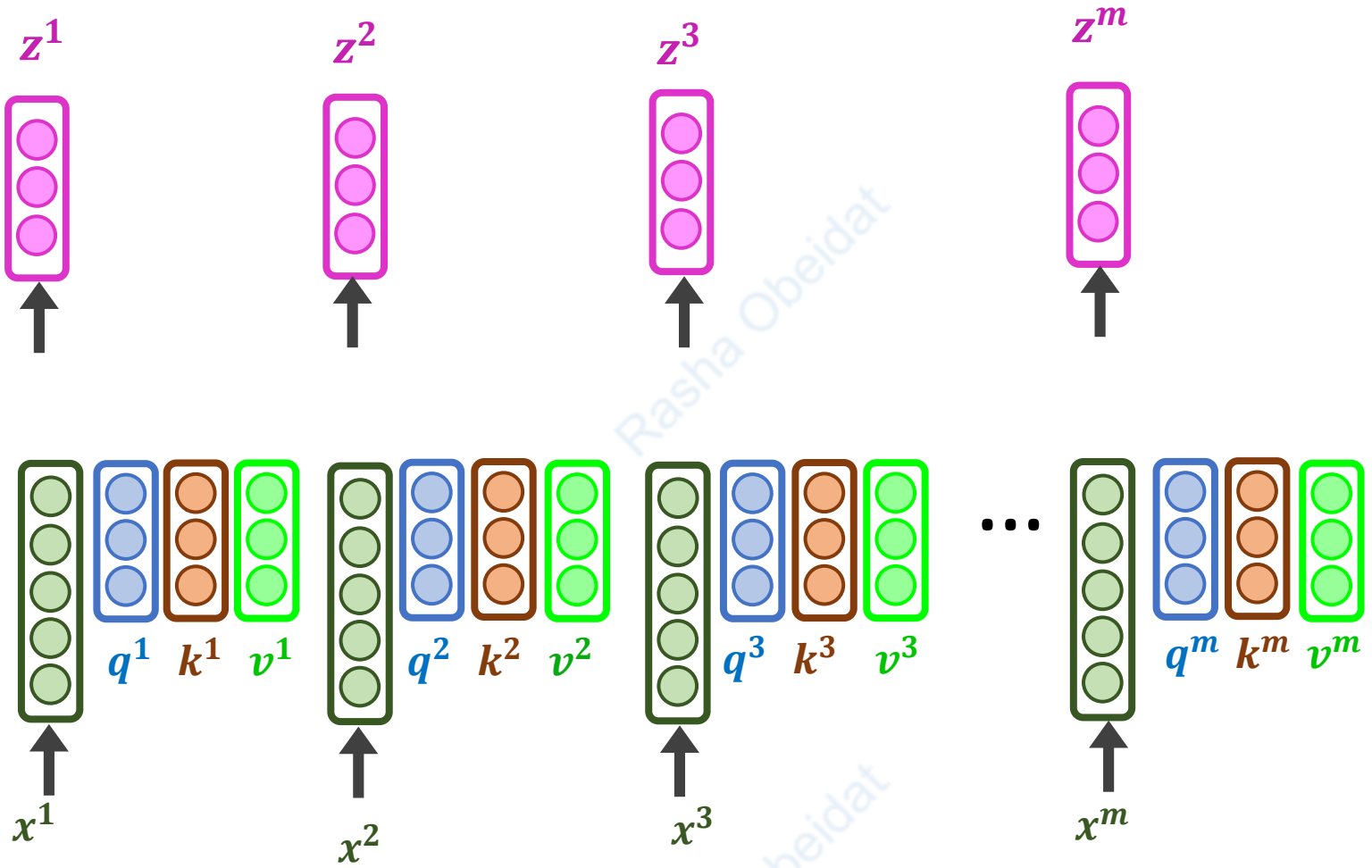
z^1 z^2 z^3 ...

$$z^m = \alpha^{m,1} . v^1 + \alpha^{m,2} . v^2 + \alpha^{m,3} . v^3 + \cdots + \alpha^{m,m} . v^m$$

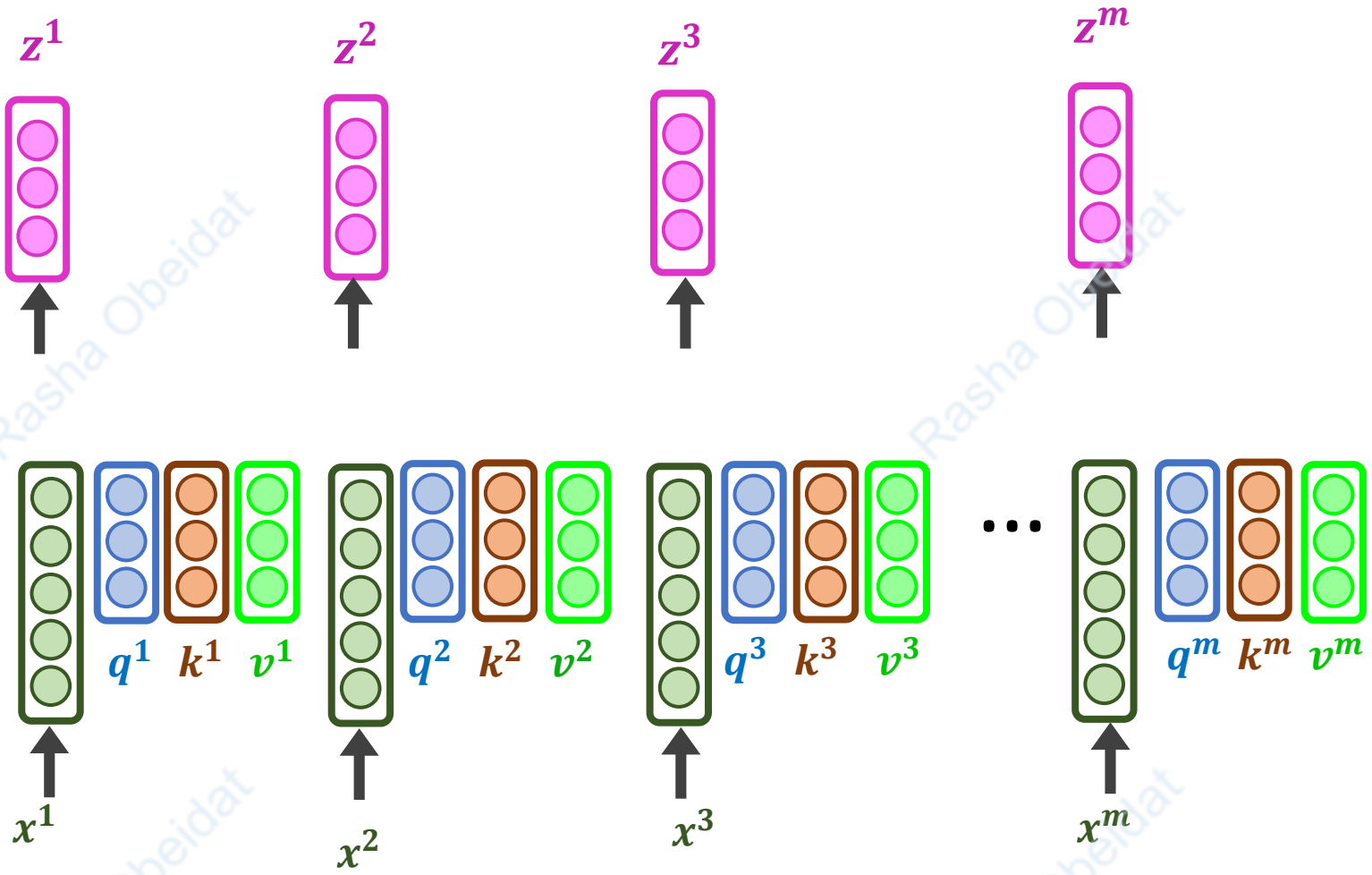




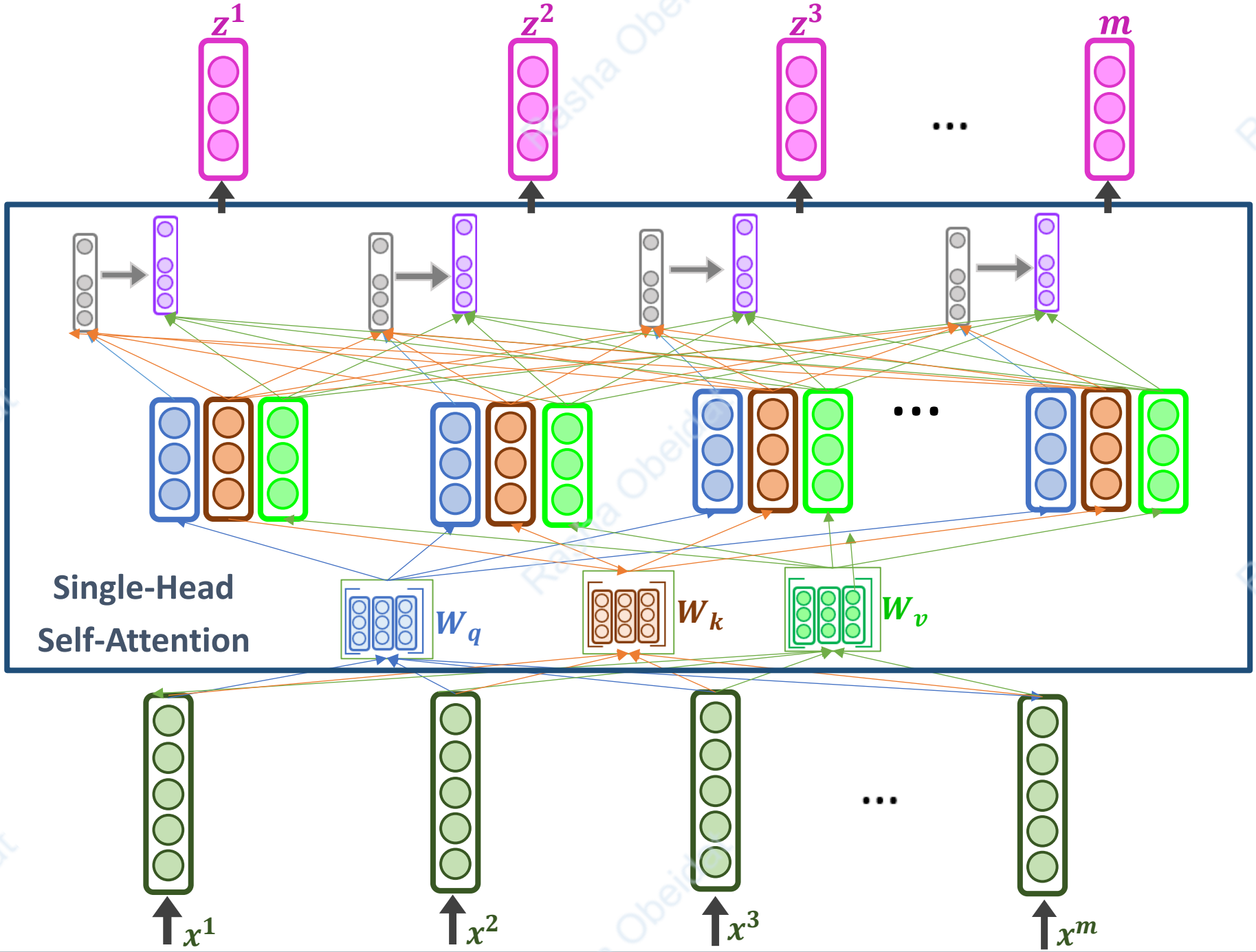
Self-Attention



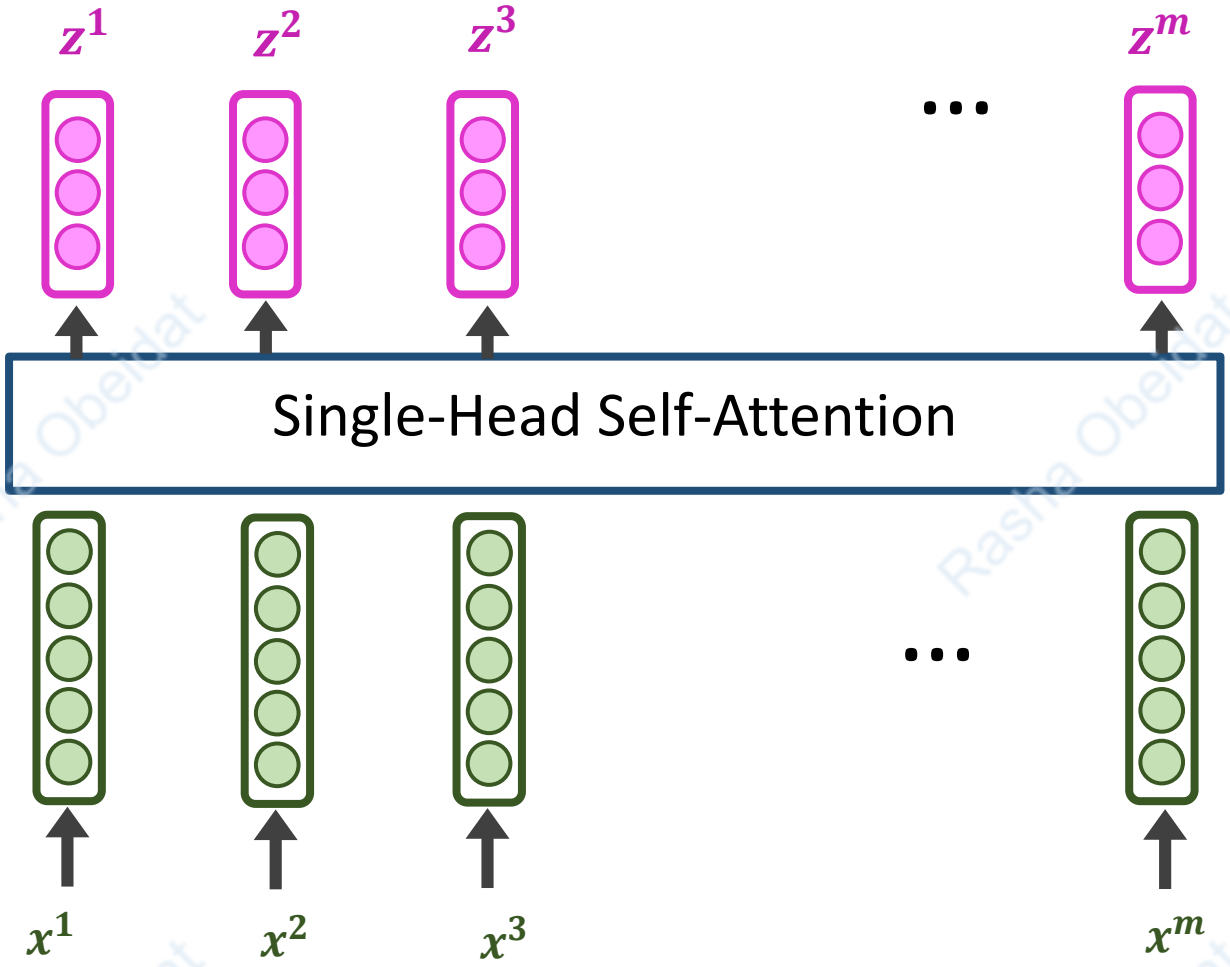
Parallelism in Self-Attention



- The outputs (z^i) for all tokens in the sequence can be computed simultaneously (in parallel).
- This is because self-attention computes relationships between all tokens at once using the keys, queries, and values, instead of processing one token at a time.
- **Parallelism** in self-attention enables much faster computation, especially for long sequences, making it more efficient than RNNs.


z^1
z^2
z^3
m
Single-Head
Self-Attention
W_q
W_k
W_v
x^1
x^2
x^3
x^m

Self-Attention



- All of what have been described is a ***Single Head self-attention unit*** !!!!

$$\textbf{Attention}\,(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \boldsymbol{\alpha}\,\boldsymbol{V}$$

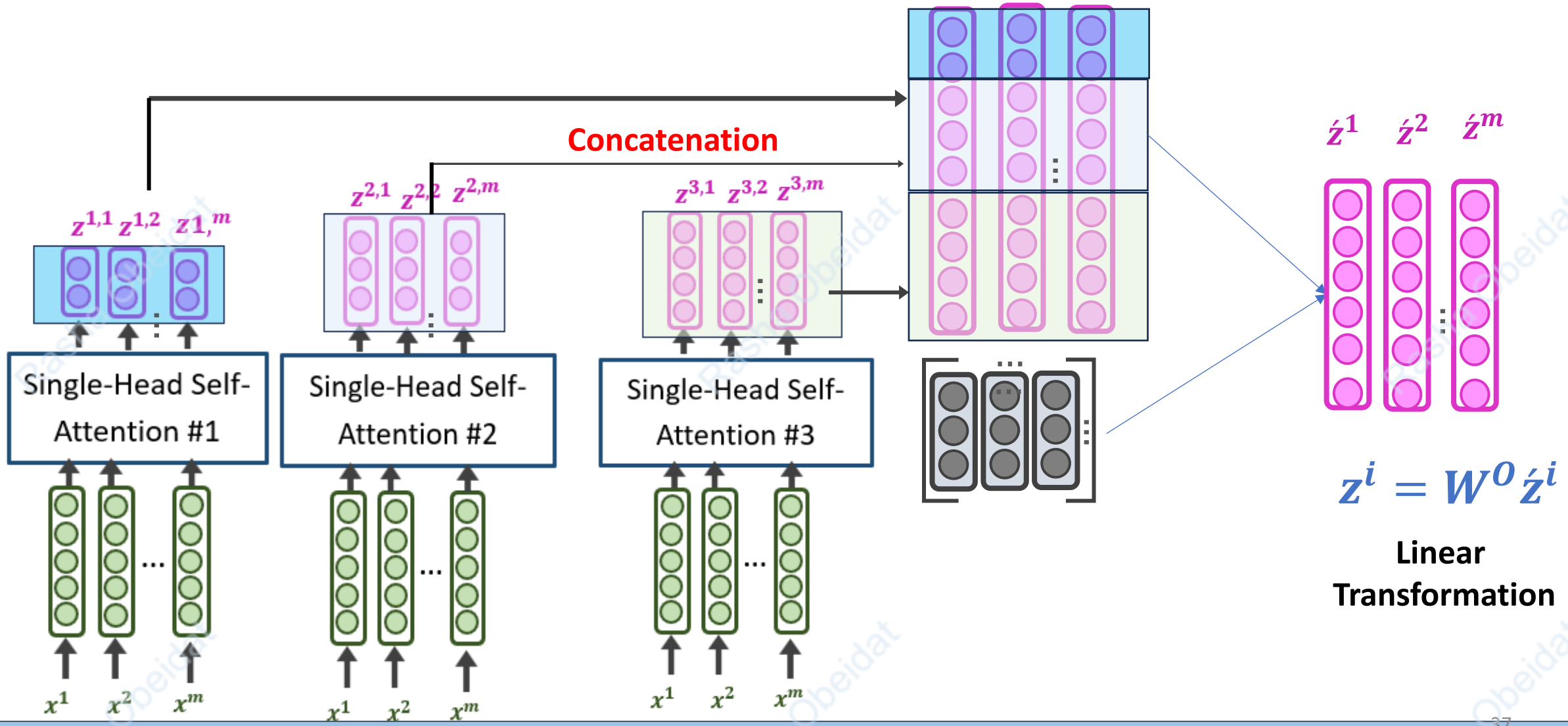
$$\textbf{Attention}\,(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \boldsymbol{Softmax}\left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d^k}}}\right)\boldsymbol{V}$$

(The notation used in the original paper: Attention is all you need!)

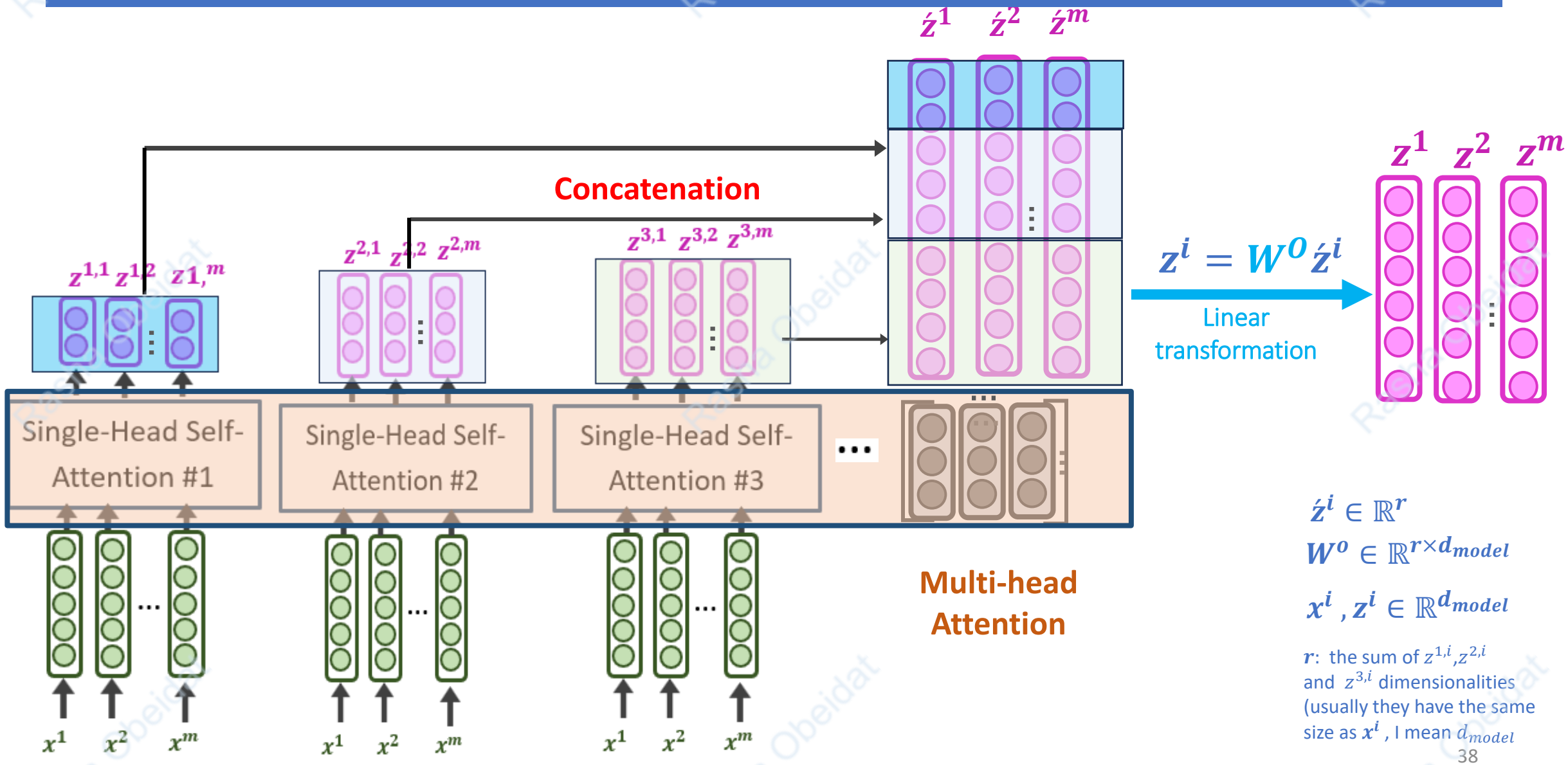
Moving from single head to multi head Self-Attention

- **Single-Head Attention**:
  - Uses one set of Query, Key, and Value matrices to compute attention.
  - Focuses on a single aspect or pattern in the data at a time.
- **Multi-Head Attention**:
  - It is found that it beneficial to linearly project the **queries**, **keys** and **values** $\boldsymbol{h}$ times with different, learned linear projections to $\boldsymbol{d^k}$, $\boldsymbol{d^k}$ and $\boldsymbol{d^v}$ dimensions, respectively.
  - On each of these projected versions of *passes* the input into multiple heads and computes attention independently for each head.
  - On each of these projected versions of queries, keys and values, multiple single-head attention are performed in parallel.
  - Each head has its own **Query**, **Key**, and **Value** matrices (probability with different dimensions (various $\boldsymbol{d^k}$ and $\boldsymbol{d^k}$ values between heads) , allowing the model to focus on different aspects of the input.
  - Outputs from all heads are ***concatenated*** and passed through a linear transformation.

Moving from single head to multi head Self-Attention



Moving from single head to multi head Self-Attention



$\acute{z}^i \in \mathbb{R}^r$

$W^O \in \mathbb{R}^{r \times d_{model}}$

$x^i, z^i \in \mathbb{R}^{d_{model}}$

r: the sum of $z^{1,i}$, $z^{2,i}$ and $z^{3,i}$ dimensionalities (usually they have the same size as x^i, I mean d_{model}

Moving from single head to multi head Self-Attention

- **Benefits of Multi-Head Attention**:
  - Captures diverse patterns and relationships in the data.
  - Improves model capacity and performance by leveraging multiple perspectives.

Rasha Obeidat

Self-Attention: Goals

- **Contextual Representation**:
  - Transform each word in a sequence into a rich, context-aware embedding.
  - Use weighted combinations of all other words in the sequence to achieve this (akin to deep CNN layers in image processing).
- **Dynamic Influence**:
  - Inspired by attention, allow each word to determine “How much should I pay attention to each of my neighbors?”
  - Compute these relationships dynamically through attention weights.

- These slides are the intellectual property of **Dr. Rasha Obeidat®**. If you wish to use them for teaching **your course**, please contact the author for permission.
  The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
  - Obeidat.rm@gmail.com
  - Rmobeidat@just.edu.jo



Rasha Obeidat